ĐẠI HỌC BÁCH KHOA HÀ NỘI

TRƯỜNG ĐIỆN – ĐIỆN TỬ



BÁO CÁO VLSI

ĐỀ TÀI: THIẾT KẾ BỘ GIẢI MÃ VITERBI

Giáo viên hướng dẫn: TS. Nguyễn Vũ Thắng

Nhóm sinh viên thực hiện: Nhóm 11

1. Nguyễn Cảnh Hoàng Sơn 20223809
2. Lê Đăng Quang 20233604
3. Thái Huy Tâm 20214083

Hà Nội, 05/2026

LỜI NÓI ĐẦU

Trong các hệ thống truyền thông số, dữ liệu truyền qua kênh thường chịu ảnh hưởng của nhiễu, suy hao và các sai lệch trong quá trình truyền dẫn. Vì vậy, các kỹ thuật mã hóa kênh được sử dụng nhằm tăng khả năng phát hiện và sửa lỗi, giúp nâng cao độ tin cậy của hệ thống. Trong số các phương pháp mã hóa kênh, mã tích chập kết hợp với thuật toán giải mã Viterbi là một hướng tiếp cận phổ biến nhờ khả năng tìm ra chuỗi dữ liệu có xác suất đúng cao nhất dựa trên mô hình trellis.

Bài tập lớn với đề tài "Thiết kế bộ giải mã Viterbi" được thực hiện nhằm tìm hiểu nguyên lý hoạt động của mã tích chập và thuật toán Viterbi, đồng thời hiện thực hóa bộ giải mã ở mức RTL bằng ngôn ngữ Verilog HDL. Thiết kế trong báo cáo hướng tới bộ giải mã cho mã tích chập có constraint length (K = 3), code rate (R = 1/2), với dữ liệu đầu vào là chuỗi mã hóa 16 bit và dữ liệu đầu ra là chuỗi giải mã 8 bit.

Trong quá trình thực hiện, nhóm tiến hành phân tích thuật toán, xây dựng kiến trúc hệ thống, thiết kế các khối chức năng như sym_extract, bmu, acsu, pmsm, smu_tbu và ctrl_unit, sau đó viết testbench để kiểm tra hoạt động của từng khối cũng như toàn bộ hệ thống. Các nội dung về mô phỏng, kiểm chứng chức năng, tổng hợp logic và layout sẽ được trình bày dựa trên kết quả thực tế thu được trong quá trình triển khai. Đối với các thông số như tần số hoạt động, diện tích, công suất, timing slack, DRC, LVS và antenna, báo cáo sẽ sử dụng kết quả từ các công cụ thiết kế khi có đầy đủ dữ liệu.

Báo cáo được trình bày theo trình tự từ cơ sở lý thuyết đến thiết kế chi tiết và kiểm chứng, nhằm thể hiện rõ mối liên hệ giữa thuật toán Viterbi và cách hiện thực phần cứng của bộ giải mã. Thông qua bài tập lớn này, nhóm có cơ hội củng cố kiến thức về thiết kế mạch số, quy trình mô tả phần cứng bằng Verilog, xây dựng testbench và bước đầu tiếp cận quy trình thiết kế VLSI.

Nhóm xin chân thành cảm ơn TS. Nguyễn Vũ Thắng đã hướng dẫn và định hướng trong quá trình thực hiện đề tài. Do thời gian và kinh nghiệm còn hạn chế, báo cáo khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được sự góp ý của thầy để hoàn thiện hơn thiết kế cũng như nội dung báo cáo.

MỤC LỤC

LỜI NÓI ĐẦU **1**

MỤC LỤC **3**

DANH MỤC HÌNH VẼ **7**

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT **11**

1.1. Tổng quan về mã hóa kênh và mã tích chập **11**

1.1.1. Vai trò của mã hóa kênh trong truyền thông số 11

1.1.2. Khái niệm mã tích chập 12

1.2. Các tham số cơ bản của mã tích chập **13**

1.2.1. Số bit đầu vào (k) và số bit đầu ra (n) 13

1.2.2. Số phần tử nhớ (m) và độ dài ràng buộc (K) 13

1.2.3. Tốc độ mã (R) 14

1.2.4. Số trạng thái của bộ mã tích chập 14

1.3. Mô hình mã tích chập sử dụng trong đề tài **15**

1.3.1. Cấu trúc bộ mã hóa với (K = 3), (R = 1/2) 15

1.3.2. Đa thức sinh ($\boldsymbol{G1 = 111}$) và ($\boldsymbol{G2 = 101}$) 15

1.3.3. Công thức tạo hai bit đầu ra 15

1.3.4. Bảng chuyển trạng thái của bộ mã hóa 16

1.3.5. Sơ đồ trạng thái của bộ mã hóa 17

1.3.6. Sơ đồ trellis của bộ mã hóa 17

1.4. Cơ sở lý thuyết thuật toán giải mã Viterbi **18**

1.4.1. Nguyên lý cực đại khả năng 19

1.4.2. Khoảng cách Hamming trong giải mã cứng 19

1.4.3. Branch metric 20

1.4.4. Path metric 21

1.4.5. Survivor path 22

1.4.6. Traceback 22

CHƯƠNG 2. THIẾT KẾ VITERBI DECODER **24**

2.1. Top module (viterbi_decoder) **24**

2.1.1. Sơ đồ khối 24

2.1.2. Bảng dữ liệu đầu ra đầu vào 24

2.1.3. Chức năng 24
2.1.4. Sơ đồ nối dây bên trong 25
2.1.5. Lưu đồ thuật toán tổng quát viterbi decoder 26
2.1.6. Timing diagram tổng quát viterbi decoder 27
2.2. Khối Symbol Extractor (sym_extract) 27
2.2.1. Sơ đồ khối 27
2.2.2. Bảng dữ liệu đầu ra đầu vào 27
2.2.3. Chức năng 28
2.2.4. Lưu đồ thuật toán 28
2.2.5. Timing diagram 29
2.3. Khối Branch Metric Unit (bmu) 29
2.3.1. Sơ đồ khối 29
2.3.2. Bảng dữ liệu đầu ra đầu vào 29
2.3.3. Chức năng 30
2.3.4. Lưu đồ thuật toán 31
2.3.5. Timing diagram 31
2.4. Khối Add-Compare-Select Unit (acsu) 32
2.4.1. Sơ đồ khối 32
2.4.2. Bảng dữ liệu đầu ra đầu vào 32
2.4.3. Chức năng 33
2.4.4. Lưu đồ thuật toán 34
2.4.5. Timing diagram 35
2.5. Khối Path Metric Store Memory (pmsm) 35
2.5.1. Sơ đồ khối 35
2.5.2. Bảng dữ liệu đầu ra đầu vào 35
2.5.3. Chức năng 36
2.5.4. Lưu đồ thuật toán 37
2.5.5. Timing diagram 37
2.6. Khối Survivor Memory and Traceback Unit (smu_tbu) 38
2.6.1. Sơ đồ khối 38
2.6.2. Bảng dữ liệu đầu ra đầu vào 38
2.6.3. Chức năng 39
2.6.4. Lưu đồ thuật toán 40

2.6.5. Timing diagram....41
2.7. Khối Control Unit (ctr_unit)....42
2.7.1. Sơ đồ khối....42
2.7.2. Bảng dữ liệu đầu ra đầu vào....42
2.7.3. Chức năng....42
2.7.4. Lưu đồ thuật toán....43
2.7.5. Timing diagram....44
CHƯƠNG 3. KẾT QUẢ MÔ PHỎNG....45
3.1. Khối sym_extract....45
3.2. Khối bmu....46
3.3. Khối acsu....47
3.4. Khối pmsm....48
3.5. Khối smu_tbu....49
3.6. Khối ctrl_unit....49
3.7. Mô phỏng toàn hệ thống viterbi_top....51
CHƯƠNG 4. THIẾT KẾ VẬT LÝ....54
4.1. Mục tiêu thiết kế....54
4.2. Tổng hợp logic (Synthesis) và chuẩn bị dữ liệu đầu vào cho layout54
4.2.1. Vai trò của giai đoạn Synthesis....54
4.2.2. Thiết kế file ràng buộc constraints.sdc....55
4.2.3. Thiết lập file chạy tự động scripts genus.tcl....56
4.3. Kết quả sau Synthesis....58
4.3.1. Timing....58
4.3.2. Power....59
4.3.3. Area....59
4.4. Thiết kế Layout....60
4.4.1. Thiết lập file viterbi_mmmc.view....60
4.4.2. Mở tool Cadence Innovus....60
4.4.3. Import Design....61
4.4.4. Floorplanning....62

4.4.5. Placement 66
4.4.6. Clock Tree System – CTS 68
4.4.7. Định tuyến chi tiết – Detail Route 68
4.4.8. Add Filler 69
4.4.9. Check DRC, Connectivity, Antenna 70
4.4.10. Xuất file GDSII/OASIS 71
4.5. Kết quả sau Layout 71
4.5.1. Timing 71
4.5.2. Power 72
4.5.3. Area 73
KẾT LUẬN 74

DANH MỤC HÌNH VẼ

Hình 1.1. Sơ đồ bộ mã hóa tích chập sử dụng đa thức sinh $G1 = 111, G2 = 101$ 15
Hình 1.2. Sơ đồ trạng thái của bộ mã hóa tích chập 17
Hình 1.3. Sơ đồ trellis của bộ mã hóa tích chập 17
Hình 2.1. Sơ đồ khối Top module 24
Hình 2.2. Sơ đồ nối dây bên trong Top module 25
Hình 2.3. Lưu đồ thuật toán tổng quát Viterbi decoder.....26
Hình 2.4.Timing diagram tổng quát Viterbi decoder 27
Hình 2.5. Sơ đồ khối của khối sym_extract.....27
Hình 2.6. Lưu đồ thuật toán của khối sym_extract.....28
Hình 2.7. Timing diagram của khối sym_extract.....29
Hình 2.8. Sơ đồ khối của khối bmu.....29
Hình 2.9. Lưu đồ thuật toán của khối bmu 31
Hình 2.10. Timing diagram của khối bmu.....31
Hình 2.11. Sơ đồ khối của khối acsu.....32
Hình 2.12. Lưu đồ thuật toán của khối acsu 34
Hình 2.13. Timing diagram của khối acsu.....35
Hình 2.14. Sơ đồ khối của khối pmsm 35
Hình 2.15. Lưu đồ thuật toán của khối pmsm 37
Hình 2.16. Timing diagram của khối pmsm 37
Hình 2.17. Sơ đồ khối của khối smu_tbu 38
Hình 2.18. (a) Lưu đồ ghi decision bit vào survivor memory (b) Lưu đồ chọn min_state cho quá trình traceback (c) Lưu đồ thực hiện traceback trong khối smu_tbu.....41
Hình 2.19. Timing diagram của khối smu_tbu 41
Hình 2.20. Sơ đồ khối của khối ctrl_unit 42
Hình 2.21. Lưu đồ thuật toán của khối ctrl_unit.....43
Hình 2.22. Timing diagram của khối ctrl_unit 44
Hình 3.1. Kết quả transcript mô phỏng khối sym_extract 45
Hình 3.2. Waveform mô phỏng khối sym_extract 46
Hình 3.3. Kết quả transcript mô phỏng khối bmu.....46

Hình 3.4. Waveform mô phỏng khối bmu 46
Hình 3.5. Transcript mô phỏng khối acsu 47
Hình 3.6. Waveform mô phỏng khối acsu 47
Hình 3.7. Waveform mô phỏng khối pmsm 48
Hình 3.8. Transcript mô phỏng khối smu_tbu 49
Hình 3.9. Waveform mô phỏng khối smu_tbu 49
Hình 3.10. Transcript mô phỏng khối ctrl_unit 50
Hình 3.11. Waveform mô phỏng khối ctrl_unit 50
Hình 3.12. Transcript mô phỏng các test cơ bản của khối viterbi_top 52
Hình 3.13. Waveform mô phỏng các test cơ bản khối viterbi_top 52
Hình 3.14. Kết quả mô phỏng top module với bộ test đọc từ file input/output ... 53
Hình 4.1. File constraints.sdc 55
Hình 4.2. File scripts genus.tcl 56
Hình 4.3. Khởi chạy Cadence Genus và thực thi script genus.tcl 58
Hình 4.4. Báo cáo timing sau synthesis 58
Hình 4.5. Báo cáo Power sau synthesis 59
Hình 4.6. Báo cáo Area sau synthesis 59
Hình 4.7. File viterbi_mmmc.view 60
Hình 4.8. Khởi chạy Cadence Innovus 60
Hình 4.9. Giao diện GUI của Innovus 60
Hình 4.10. Import Design 61
Hình 4.11. Kết quả sau khi Import Design thành công 61
Hình 4.12. Setup cấu hình Floorplanning 62
Hình 4.13. Kết quả sau khi Floorplanning 62
Hình 4.14. Cấu hình các pin của chip 63
Hình 4.15. Thực hiện kiểm tra check_pin_assignment 63
Hình 4.16. Add Ring 63
Hình 4.17. Kết quả sau khi Add Ring 64
Hình 4.18. Add Stripes 64
Hình 4.19. Kết quả sau khi Add Stripes 65
Hình 4.20. Kết nối chân VDD và VSS của mọi linh kiện 65
Hình 4.21. Add Route 65

Hình 4.22. Kết quả sau khi Add Route.....66
Hình 4.23. Thực hiện lệnh place_opt_design 67
Hình 4.24. Kết quả sau khi Placement 67
Hình 4.25. Tổng hợp và tối ưu CTS.....68
Hình 4.26. Cấu hình NanoRoute.....68
Hình 4.27. Kết quả sau khi NanoRoute.....69
Hình 4.28. Add Filler.....69
Hình 4.29. Kết quả sau khi Adder Filler.....70
Hình 4.30. Kết quả check_drc.....70
Hình 4.31. Kết quả check_connectivity 70
Hình 4.32. Kết quả check_antenna 71
Hình 4.33. Xuất file GDSII/OASIS 71
Hình 4.34. Báo cáo Timing sau Layout.....71
Hình 4.35. Báo cáo Power sau Layout 72
Hình 4.36. Báo cáo Area sau Layout.....73

DANH MỤC BẢNG BIỂU

Bảng 1.1. Bảng chuyển trạng thái của bộ mã hóa tích chập 16
Bảng 1.2. Vai trò của các thành phần trellis trong giải mã Viterbi..... 18
Bảng 1.3. Ý nghĩa khoảng cách Hamming trong giải mã cứng 20
Bảng 2.1. Bảng dữ liệu I/O của Top module..... 24
Bảng 2.2. Bảng dữ liệu I/O của khối sym_extract 27
Bảng 2.3. Bảng dữ liệu I/O của khối bmu 29
Bảng 2.4. Bảng dữ liệu I/O của khối acsu 32
Bảng 2.5. Bảng dữ liệu I/O của khối pmsm..... 35
Bảng 2.6. Bảng dữ liệu I/O của khối smu_tbu 38
Bảng 2.7. Bảng dữ liệu I/O của khối ctrl_unit 42
Bảng 3.1. Testcase kiểm chứng khối sym_extract..... 45
Bảng 3.2. Testcase kiểm chứng khối bmu..... 46
Bảng 3.3. Testcase kiểm chứng khối acsu..... 47
Bảng 3.4. Testcase kiểm chứng khối pmsm..... 48
Bảng 3.5. Testcase kiểm chứng khối smu_tbu 49
Bảng 3.6. Testcase kiểm chứng khối ctrl_unit 49
Bảng 3.7. Testcase cơ bản kiểm chứng khối viterbi_top 51

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT

1.1. Tổng quan về mã hóa kênh và mã tích chập

1.1.1. Vai trò của mã hóa kênh trong truyền thông số

Trong các hệ thống truyền thông số, dữ liệu được truyền từ phía phát đến phía thu dưới dạng chuỗi bit nhị phân. Tuy nhiên, trong quá trình truyền qua kênh truyền, tín hiệu có thể bị ảnh hưởng bởi nhiễu, suy hao, méo tín hiệu hoặc các tác động không mong muốn từ môi trường. Các yếu tố này có thể làm một hoặc nhiều bit trong chuỗi dữ liệu bị sai lệch so với dữ liệu ban đầu.

Để giảm ảnh hưởng của lỗi bit và nâng cao độ tin cậy của hệ thống, kỹ thuật mã hóa kênh được sử dụng trước khi truyền dữ liệu. Mã hóa kênh bổ sung thêm các bit dư có quy luật vào chuỗi thông tin gốc. Các bit dư này không làm thay đổi nội dung thông tin cần truyền, nhưng tạo ra cấu trúc dữ liệu giúp phía thu có thể phát hiện và sửa lỗi trong một phạm vi nhất định.

Về mặt tổng quát, quá trình mã hóa kênh có thể xem là phép biến đổi từ chuỗi thông tin đầu vào sang chuỗi mã hóa đầu ra. Gọi chuỗi thông tin gốc là:

$$u = (u_0, u_1, u_2, \dots, u_{L-1})$$

và chuỗi sau mã hóa là:

$$c = (c_0, c_1, c_2, \dots, c_{N-1})$$

Trong đó, L là số bit dữ liệu gốc và N là số bit sau mã hóa. Thông thường, do có thêm các bit dư phục vụ kiểm soát lỗi nên:

$$N > L$$

Tỷ lệ giữa số bit thông tin gốc và số bit sau mã hóa được gọi là tốc độ mã, ký hiệu là R:

$$R = \frac{L}{N}$$

Tốc độ mã càng nhỏ thì lượng bit dư được thêm vào càng nhiều, khả năng chống lỗi có thể tốt hơn nhưng số bit cần truyền cũng tăng lên. Ngược lại, tốc độ mã càng lớn thì dữ liệu truyền đi ngắn hơn, nhưng khả năng sửa lỗi có thể giảm. Vì vậy, trong thiết kế hệ thống truyền thông, mã hóa kênh cần cân bằng giữa độ tin cậy truyền dữ liệu, tốc độ truyền và độ phức tạp phần cứng.

Trong số các phương pháp mã hóa kênh, mã tích chập là một dạng mã được sử dụng phổ biến do có khả năng xử lý chuỗi dữ liệu liên tục và có thể kết hợp hiệu quả với thuật toán giải mã Viterbi.

1.1.2. Khái niệm mã tích chập

Mã tích chập, hay *Convolutional Code*, là một loại mã sửa lỗi có bộ nhớ. Đặc điểm quan trọng của mã tích chập là đầu ra tại một thời điểm không chỉ phụ thuộc vào bit đầu vào hiện tại mà còn phụ thuộc vào một số bit đầu vào trước đó. Các bit trước đó được lưu lại trong các phần tử nhớ, thường được hiện thực bằng các thanh ghi dịch.

Gọi bit đầu vào tại thời điểm i là u_i, các bit trước đó lần lượt là $u_{i-1}, u_{i-2}, \ldots, u_{i-m}$, trong đó m là số phần tử nhớ của bộ mã hóa. Khi đó, một bit đầu ra của mã tích chập có thể được mô tả tổng quát như sau:

$$c_i = f(u_i, u_{i-1}, u_{i-2}, \ldots, u_{i-m})$$

Trong đó, (f) là hàm logic xác định bởi cấu trúc bộ mã hóa. Với mã tích chập nhị phân, hàm (f) thường được xây dựng từ các phép XOR giữa bit hiện tại và các bit đang được lưu trong thanh ghi dịch.

Nếu bộ mã hóa có (n) đầu ra, tại mỗi thời điểm sẽ tạo ra (n) bit mã hóa. Mỗi bit đầu ra được xác định bởi một đa thức sinh riêng. Dạng tổng quát của bit đầu ra thứ (j) tại thời điểm (i) có thể viết như sau:

$$v_i^{(j)} = \sum_{l=0}^{m} g_l^{(j)} u_{i-l} \pmod 2$$

Trong đó:

- $v_i^{(j)}$ là bit đầu ra thứ j tại thời điểm i;
- u_{i-l} là bit đầu vào hiện tại hoặc các bit trước đó;
- $g_l^{(j)}$ là hệ số của đa thức sinh thứ j, có giá trị 0 hoặc 1;
- phép cộng theo modulo 2 tương đương với phép XOR trong mạch số;
- (m) là số phần tử nhớ của bộ mã hóa.

Công thức trên cho thấy đầu ra của mã tích chập được tạo ra bằng cách kết hợp có chọn lọc các bit đầu vào hiện tại và quá khứ. Nếu hệ số $g_l^{(j)} = 1$, bit u_{i-l} tham gia vào phép XOR để tạo đầu ra. Nếu $g_l^{(j)} = 0$, bit đó không ảnh hưởng đến đầu ra tương ứng.

Cấu trúc cơ bản của một bộ mã hóa tích chập gồm ba thành phần chính: chuỗi bit đầu vào, các thanh ghi dịch dùng để lưu trạng thái và các mạch XOR tạo bit đầu ra theo các đa thức sinh. Nhờ có các phần tử nhớ, một bit thông tin đầu vào có thể ảnh hưởng đến nhiều bit mã hóa đầu ra ở các thời điểm khác nhau. Đây chính là lý do mã tích chập có khả năng tạo ra sự dư thừa có cấu trúc trong chuỗi dữ liệu.

1.2. Các tham số cơ bản của mã tích chập

1.2.1. Số bit đầu vào (k) và số bit đầu ra (n)

Trong một chu kỳ mã hóa, bộ mã tích chập nhận vào k bit thông tin và tạo ra n bit mã hóa ở đầu ra. Hai tham số này quyết định trực tiếp lượng dữ liệu đầu ra được sinh ra từ dữ liệu đầu vào.

$$k \text{ bit đầu vào} \rightarrow n \text{ bit đầu ra}$$

Trong đề tài này, bộ mã hóa tương ứng với bộ giải mã cần thiết kế có:

$$k = 1$$

$$n = 2$$

Điều đó có nghĩa là mỗi bit dữ liệu gốc sau khi mã hóa sẽ tạo thành một symbol gồm 2 bit. Vì vậy, ở phía giải mã, dữ liệu mã hóa 16 bit sẽ được chia thành 8 symbol, mỗi symbol có độ dài 2 bit.

1.2.2. Số phần tử nhớ (m) và độ dài ràng buộc (K)

Mã tích chập là loại mã có bộ nhớ, do đó đầu ra không chỉ phụ thuộc vào bit đầu vào hiện tại mà còn phụ thuộc vào một số bit trước đó. Số bit trước đó được lưu lại trong các thanh ghi dịch được gọi là số phần tử nhớ, ký hiệu là m.

Độ dài ràng buộc, ký hiệu là K, cho biết số lượng bit đầu vào có thể ảnh hưởng đến đầu ra tại một thời điểm. Độ dài ràng buộc bao gồm bit hiện tại và các bit đang được lưu trong bộ nhớ. Do đó, quan hệ giữa K và m được xác định bởi:

$$K = m + 1$$

Với yêu cầu của bài toán:

$$K = 3$$

nên số phần tử nhớ của bộ mã hóa là:

$$m = K - 1 = 3 - 1 = 2$$

Như vậy, bộ mã hóa có hai phần tử nhớ. Tại mỗi thời điểm, đầu ra của bộ mã hóa phụ thuộc vào bit đầu vào hiện tại và hai bit trước đó. Đây là cơ sở để hình

thành các trạng thái của bộ mã hóa và xây dựng sơ đồ trellis cho thuật toán giải mã Viterbi.

1.2.3. Tốc độ mã (R)

Tốc độ mã, ký hiệu là (R), biểu thị tỷ lệ giữa số bit thông tin đầu vào và số bit mã hóa đầu ra trong mỗi chu kỳ. Tốc độ mã được xác định theo công thức:

$$R = \frac{k}{n}$$

Trong đó:

- k là số bit đầu vào trong một chu kỳ mã hóa;
- n là số bit đầu ra trong một chu kỳ mã hóa.

Với cấu hình của đề tài:

$$k = 1, \quad n = 2$$

nên tốc độ mã là:

$$R = \frac{1}{2}$$

Điều này có nghĩa là chuỗi sau mã hóa có độ dài gấp đôi chuỗi dữ liệu ban đầu. Ví dụ, nếu chuỗi thông tin gốc có 8 bit thì chuỗi mã hóa tương ứng sẽ có 16 bit. Đây cũng là lý do bài toán yêu cầu bộ giải mã nhận đầu vào 16 bit mã hóa và khôi phục đầu ra 8 bit thông tin.

1.2.4. Số trạng thái của bộ mã tích chập

Trạng thái của bộ mã hóa tích chập được xác định bởi các bit đang lưu trong các phần tử nhớ. Nếu bộ mã hóa có m phần tử nhớ, mỗi phần tử nhớ có thể nhận một trong hai giá trị 0 hoặc 1. Do đó, số trạng thái có thể có của bộ mã hóa là:

$$N_{\text{state}} = 2^m$$

Do:

$$m = K - 1$$

nên có thể viết:

$$N_{\text{state}} = 2^{K-1}$$

Với (K = 3), ta có:

$$N_{\text{state}} = 2^{3-1} = 2^2 = 4$$

Vì vậy, bộ mã hóa trong đề tài có bốn trạng thái. Các trạng thái này được biểu diễn bằng hai bit như sau:

$$00, 01, 10, 11$$

Bốn trạng thái trên sẽ được sử dụng để xây dựng bảng chuyển trạng thái, sơ đồ trạng thái và sơ đồ trellis. Trong quá trình giải mã Viterbi, thuật toán sẽ xét các đường đi có thể xảy ra qua bốn trạng thái này để tìm ra chuỗi dữ liệu gốc có khả năng cao nhất.

1.3. Mô hình mã tích chập sử dụng trong đề tài

1.3.1. Cấu trúc bộ mã hóa với (K = 3), (R = 1/2)

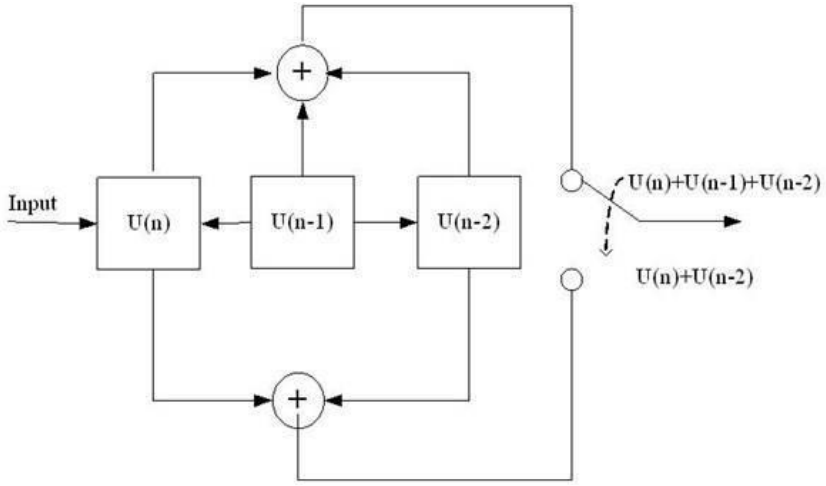
- **Constraint length:** $K = 3 \Rightarrow m = 2$
- **Code rate:** $R = \frac{1}{2} \Rightarrow k = 1, n = 2$

1.3.2. Đa thức sinh $(\boldsymbol{G_1 = 111})$ *và* $(\boldsymbol{G_2 = 101})$

Sử dụng hai đa thức sinh:

$$G_1(x) = 1 + x + x^2$$

$$G_2(x) = 1 + x^2$$



Hình 1.1. Sơ đồ bộ mã hóa tích chập sử dụng đa thức sinh $G_1 = 111, G_2 = 101$

Đa thức $G_1 = 111_2$ cho biết cả ba bit u_n, u_{n-1} và u_{n-2} đều tham gia tạo bit đầu ra thứ nhất. Trong khi đó, đa thức $G_2 = 101_2$ cho biết bit hiện tại u_n và bit trễ hai chu kỳ u_{n-2} tham gia tạo bit đầu ra thứ hai, còn bit u_{n-1} không tham gia.

1.3.3. Công thức tạo hai bit đầu ra

Từ hai đa thức sinh G_1 và G_2, hai bit đầu ra tại thời điểm n được xác định bởi:

$$v_1 = u_n \oplus u_{n-1} \oplus u_{n-2}$$

$$v_2 = u_n \oplus u_{n-2}$$

Trong đó, $\oplus$ là phép XOR. Cặp bit đầu ra tại thời điểm n là:

$$v_n = (v_1 v_2)$$

Ví dụ, nếu tại một thời điểm có:

$$u_n = 1,\ u_{n-1} = 0,\ u_{n-2} = 1$$

thì hai bit đầu ra được tính như sau:

$$v_1 = 1 \oplus 0 \oplus 1 = 0$$

$$v_2 = 1 \oplus 1 = 0$$

Do đó, symbol đầu ra tương ứng là:

$$v_n = 00$$

1.3.4. Bảng chuyển trạng thái của bộ mã hóa

Trạng thái của bộ mã hóa được xác định bởi hai bit đang lưu trong thanh ghi dịch. Trạng thái hiện tại được ký hiệu dưới dạng:

$$S = (u_{n-1}u_{n-2})$$

Do có hai bit nhớ, bộ mã hóa có bốn trạng thái:

$$00, 01, 10, 11$$

Với mỗi trạng thái hiện tại, bit đầu vào có thể nhận giá trị 0 hoặc 1. Từ đó, bộ mã hóa tạo ra một symbol đầu ra 2 bit và chuyển sang trạng thái kế tiếp. Theo quy tắc dịch thanh ghi, trạng thái kế tiếp được xác định bởi:

$$S' = (u_n u_{n-1})$$

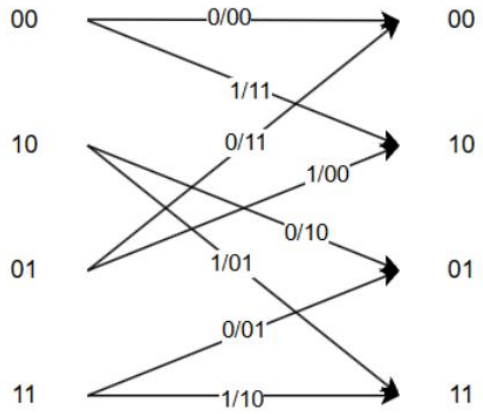
Bảng chuyển trạng thái của bộ mã hóa:

Bảng 1.1. Bảng chuyển trạng thái của bộ mã hóa tích chập

Bit đầu vào u_n	Trạng thái hiện tại $S = (u_{n-1}u_{n-2})$	Đầu ra $(v_1 v_2)$	Trạng thái kế tiếp $S' = (u_n u_{n-1})$
0	00	00	00
1	00	11	10
0	01	11	00
1	01	00	10
0	10	10	01
1	10	01	11
0	11	01	01
1	11	10	11

1.3.5. Sơ đồ trạng thái của bộ mã hóa

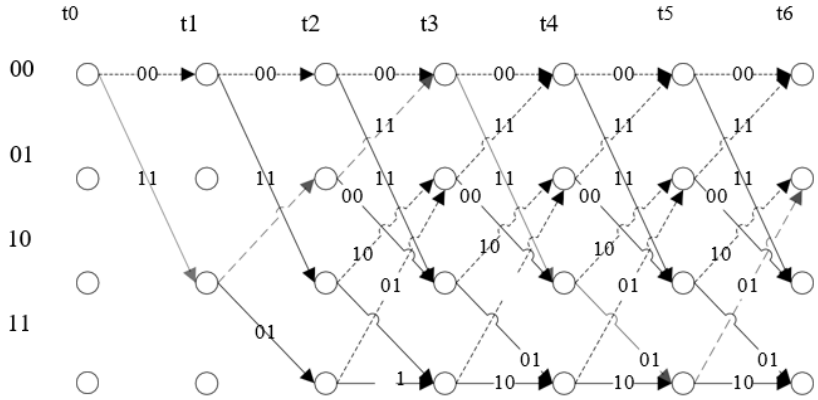
Từ bảng chuyển trạng thái ta xây dựng được sơ đồ trạng thái như sau:



Hình 1.2. Sơ đồ trạng thái của bộ mã hóa tích chập

Sơ đồ trạng thái giúp quan sát quan hệ giữa các trạng thái một cách trực quan. Tuy nhiên, để mô tả quá trình mã hóa và giải mã theo thời gian, cần mở rộng sơ đồ trạng thái thành sơ đồ trellis.

1.3.6. Sơ đồ trellis của bộ mã hóa



Hình 1.3. Sơ đồ trellis của bộ mã hóa tích chập

Sơ đồ trellis là dạng biểu diễn theo thời gian của sơ đồ trạng thái. Nếu sơ đồ trạng thái cho biết các khả năng chuyển trạng thái tổng quát, thì sơ đồ trellis cho thấy các chuyển trạng thái đó xảy ra như thế nào qua từng bước xử lý symbol.

Với chuỗi mã hóa đầu vào dài 16 bit và tốc độ mã (R = 1/2), dữ liệu đầu vào của bộ giải mã được chia thành 8 symbol 2 bit. Vì vậy, quá trình giải mã cần xét trellis trong 8 bước thời gian:

16 bit mã hóa $\Rightarrow$ 8 symbol

$$r_0, r_1, r_2, \ldots, r_7$$

Trong đó, mỗi r_i là một symbol gồm 2 bit. Tại mỗi bước i, symbol nhận được r_i được so sánh với các symbol lý thuyết trên các nhánh trellis để tính chi phí nhánh, hay branch metric.

Vai trò của trellis trong giải mã Viterbi có thể tóm tắt như sau:

Bảng 1.2. Vai trò của các thành phần trellis trong giải mã Viterbi

Thành phần trong trellis	**Ý nghĩa trong giải mã**
Nút trạng thái	Biểu diễn trạng thái có thể có của bộ mã hóa tại một thời điểm
Nhánh chuyển trạng thái	Biểu diễn một khả năng bit đầu vào và symbol đầu ra tương ứng
Symbol trên nhánh	Dùng để so sánh với symbol nhận được
Đường đi trên trellis	Tương ứng với một chuỗi bit đầu vào có thể xảy ra
Đường có metric nhỏ nhất	Được chọn làm đường có khả năng cao nhất

Thuật toán Viterbi không chọn nhánh tốt nhất tại từng thời điểm một cách độc lập. Thay vào đó, thuật toán xét chi phí tích lũy của các đường đi trên trellis. Tại mỗi trạng thái, nếu có nhiều đường đi hội tụ, thuật toán chỉ giữ lại đường có tổng chi phí nhỏ hơn. Đường được giữ lại này gọi là survivor path.

Nhờ cách biểu diễn trellis, bài toán giải mã chuỗi bit được chuyển thành bài toán tìm đường đi có tổng độ sai khác nhỏ nhất. Đây chính là cơ sở để triển khai các bước tính branch metric, cập nhật path metric, chọn survivor path và traceback trong thuật toán Viterbi.

1.4. Cơ sở lý thuyết thuật toán giải mã Viterbi

Thuật toán Viterbi là một thuật toán giải mã thường được sử dụng cho mã tích chập. Về bản chất, thuật toán này tìm đường đi có khả năng cao nhất trên sơ đồ trellis dựa trên chuỗi symbol nhận được ở đầu vào bộ giải mã. Trong trường hợp giải mã cứng, mức độ phù hợp giữa symbol nhận được và symbol lý thuyết được đánh giá bằng khoảng cách Hamming.

Đối với mã tích chập, mỗi chuỗi bit thông tin đầu vào tương ứng với một đường đi trên trellis. Khi dữ liệu truyền qua kênh bị lỗi, chuỗi symbol nhận được

có thể không trùng hoàn toàn với một đường đi hợp lệ nào trên trellis. Khi đó, thuật toán Viterbi sẽ tìm đường đi có tổng độ sai khác nhỏ nhất so với chuỗi nhận được. Đường đi này được xem là đường có khả năng cao nhất đã tạo ra chuỗi mã hóa ở phía phát.

1.4.1. Nguyên lý cực đại khả năng

Giải mã Viterbi dựa trên nguyên lý cực đại khả năng, hay *Maximum Likelihood*. Theo nguyên lý này, trong tập hợp các chuỗi dữ liệu có thể tạo ra chuỗi mã hóa nhận được, bộ giải mã sẽ chọn chuỗi có xác suất xảy ra cao nhất.

Giả sử chuỗi symbol nhận được là:

$$\boldsymbol{r} = (r_0, r_1, \ldots, r_{L-1})$$

và một chuỗi symbol lý thuyết trên trellis là:

$$\boldsymbol{v} = (v_0, v_1, \ldots, v_{L-1})$$

Nhiệm vụ của bộ giải mã là tìm chuỗi $\boldsymbol{v}$ sao cho mức độ sai khác giữa $\boldsymbol{r}$ và $\boldsymbol{v}$ là nhỏ nhất. Với giải mã cứng, tiêu chí này có thể biểu diễn thông qua tổng khoảng cách Hamming:

$$D(\boldsymbol{r}, \boldsymbol{v}) = \sum_{i=0}^{L-1} d_H(r_i, v_i)$$

Trong đó:

- $D(\boldsymbol{r}, \boldsymbol{v})$ là tổng độ sai khác giữa chuỗi nhận được và chuỗi lý thuyết;
- $d_H(r_i, v_i)$ là khoảng cách Hamming giữa symbol nhận được r_i và symbol lý thuyết v_i;
- L là số symbol được xử lý.

Đường đi được chọn là đường có tổng metric nhỏ nhất:

$$\boldsymbol{v}_{opt} = argmin_v\, D(\boldsymbol{r}, \boldsymbol{v})$$

Công thức trên thể hiện ý tưởng chính của thuật toán Viterbi: thay vì quyết định từng symbol riêng lẻ, thuật toán xét toàn bộ đường đi trên trellis và chọn đường có tổng sai khác nhỏ nhất.

1.4.2. Khoảng cách Hamming trong giải mã cứng

Trong giải mã cứng, mỗi bit nhận được chỉ có một trong hai giá trị 0 hoặc 1. Vì vậy, độ sai khác giữa symbol nhận được và symbol lý thuyết có thể được đo bằng khoảng cách Hamming.

Khoảng cách Hamming giữa hai chuỗi bit cùng độ dài là số vị trí bit khác nhau giữa hai chuỗi đó. Với hai symbol 2 bit:

$$a = a_1 a_0$$

$$b = b_1 b_0$$

khoảng cách Hamming được tính bởi:

$$d_H(a,b) = (a_1 \oplus b_1) + (a_0 \oplus b_0)$$

Trong đó, $\oplus$ là phép XOR. Vì mỗi symbol trong đề tài có độ dài 2 bit, giá trị khoảng cách Hamming chỉ có thể là 0, 1 hoặc 2.

Bảng 1.3. Ý nghĩa khoảng cách Hamming trong giải mã cứng

Khoảng cách Hamming	Ý nghĩa
0	Hai symbol giống nhau hoàn toàn
1	Hai symbol khác nhau tại một vị trí bit
2	Hai symbol khác nhau tại cả hai vị trí bit

Ví dụ, nếu symbol nhận được là (01), còn symbol lý thuyết là (11), ta có:

$$d_H(01,11) = (0 \oplus 1) + (1 \oplus 1) = 1 + 0 = 1$$

Khoảng cách Hamming càng nhỏ thì symbol lý thuyết càng gần với symbol nhận được. Đây là cơ sở để tính branch metric trong thuật toán Viterbi.

1.4.3. Branch metric

Branch metric là chi phí gán cho một nhánh trên trellis tại một thời điểm. Chi phí này biểu diễn mức độ khác nhau giữa symbol nhận được từ đầu vào bộ giải mã và symbol lý thuyết tương ứng với nhánh đang xét.

Gọi symbol nhận được tại thời điểm i là r_i, symbol lý thuyết của nhánh từ trạng thái S sang trạng thái P là $v(S \rightarrow P)$. Khi đó, branch metric của nhánh được xác định bởi:

$$BM_i(S \rightarrow P) = d_H(r_i, v(S \rightarrow P))$$

Trong đó:

- $BM_i(S \rightarrow P)$ là branch metric tại thời điểm i;
- (S) là trạng thái trước đó;
- (P) là trạng thái hiện tại hoặc trạng thái đích;
- $v(S \rightarrow P)$ là symbol lý thuyết trên nhánh chuyển trạng thái;
- d_H là khoảng cách Hamming.

Với mã tích chập (R = 1/2), mỗi symbol có 2 bit. Do đó, branch metric có thể nhận các giá trị:

$$BM_i \in (0,1,2)$$

Branch metric không quyết định ngay chuỗi dữ liệu đầu ra, mà chỉ cho biết chi phí tức thời của từng nhánh. Để chọn đường đi tốt nhất, thuật toán cần cộng dồn các branch metric theo thời gian để tạo thành path metric.

1.4.4. Path metric

Path metric là chi phí tích lũy của một đường đi trên trellis. Tại mỗi thời điểm, path metric của một trạng thái được cập nhật bằng cách cộng path metric của trạng thái trước đó với branch metric của nhánh đi đến trạng thái đang xét.

Giả sử tại thời điểm i, trạng thái P có hai nhánh đi vào từ hai trạng thái trước đó là S_0 và S_1. Khi đó, hai path metric ứng viên được tính như sau:

$$PM_i^{(0)}(P) = PM_{i-1}(S_0) + BM_i(S_0 \to P)$$

$$PM_i^{(1)}(P) = PM_{i-1}(S_1) + BM_i(S_1 \to P)$$

Path metric mới của trạng thái (P) được chọn là giá trị nhỏ hơn trong hai giá trị ứng viên:

$$PM_i(P) = min\ (\ PM_i^{(0)}(P), PM_i^{(1)}(P)\ )$$

Trong đó:

- $PM_i(P)$ là path metric của trạng thái P tại thời điểm i;
- $PM_{i-1}(S_0)$, $PM_{i-1}(S_1)$ là path metric của các trạng thái trước đó;
- BM_i là branch metric tại thời điểm hiện tại.

Quy tắc cập nhật path metric giúp thuật toán Viterbi loại bỏ dần các đường đi có tổng chi phí lớn. Thay vì lưu toàn bộ các đường đi trên trellis, thuật toán chỉ giữ lại đường tốt nhất đi đến mỗi trạng thái.

Trong trường hợp hai path metric ứng viên bằng nhau, bộ giải mã có thể sử dụng một quy ước cố định để chọn một trong hai nhánh. Việc đặt quy ước chọn khi hòa giúp phần cứng hoạt động xác định và tránh trạng thái không rõ ràng.

1.4.5. Survivor path

Survivor path là đường đi được giữ lại sau bước so sánh path metric tại mỗi trạng thái. Khi có nhiều đường đi cùng hội tụ về một trạng thái, thuật toán Viterbi chỉ giữ lại đường có path metric nhỏ nhất. Đường còn lại bị loại bỏ vì có tổng chi phí lớn hơn, tức là ít phù hợp hơn với chuỗi symbol nhận được.

Đối với mã tích chập có bốn trạng thái, tại mỗi thời điểm thuật toán cần chọn survivor path cho từng trạng thái:

$$00, 01, 10, 11$$

Mỗi trạng thái sẽ có một quyết định survivor tương ứng. Các quyết định này cần được lưu lại theo thời gian để phục vụ giai đoạn truy vết ngược. Như vậy, survivor path không chỉ là kết quả của phép so sánh tại một thời điểm, mà còn là thông tin lịch sử giúp xác định đường đi tối ưu sau khi đã xử lý toàn bộ chuỗi symbol.

Có thể hiểu quá trình chọn survivor path như sau:

Nhiều đường đi vào cùng một trạng thái $\Rightarrow$ So sánh path metric

$\Rightarrow$ Giữ lại đường có metric nhỏ nhất

Nhờ cơ chế này, số đường đi cần xét không tăng theo cấp số nhân khi độ dài chuỗi đầu vào tăng. Đây là ưu điểm quan trọng giúp thuật toán Viterbi phù hợp để triển khai trong phần cứng số.

1.4.6. Traceback

Sau khi xử lý toàn bộ các symbol đầu vào, bộ giải mã cần khôi phục chuỗi bit thông tin ban đầu. Lúc này, các path metric cuối cùng cho biết trạng thái kết thúc nào có tổng chi phí nhỏ nhất. Trạng thái bắt đầu traceback thường được chọn theo quy tắc:

$$S_{\text{end}} = argmin_S \, PM_L(S)$$

Trong đó:

- S_{end} là trạng thái kết thúc được chọn để bắt đầu truy vết;
- $PM_L(S)$ là path metric của trạng thái S sau khi xử lý L symbol;
- (L) là số bước xử lý trên trellis.

Từ trạng thái kết thúc này, bộ giải mã lần ngược lại các quyết định survivor đã lưu ở các thời điểm trước đó. Tại mỗi bước truy vết, trạng thái trước đó được

xác định dựa trên thông tin survivor path. Đồng thời, bit đầu vào tương ứng với nhánh chuyển trạng thái đó được khôi phục.

Quá trình traceback có thể mô tả tổng quát như sau:

$$S_i \rightarrow S_{i-1} \rightarrow S_{i-2} \rightarrow \ldots \rightarrow S_0$$

Với mỗi chuyển trạng thái ngược, bộ giải mã xác định được một bit dữ liệu gốc. Sau khi truy vết đủ số bước cần thiết, chuỗi bit đầu ra được tạo thành.

Đối với bài toán trong đề tài, chuỗi mã hóa đầu vào gồm 16 bit và được chia thành 8 symbol. Vì vậy, sau khi hoàn tất quá trình xử lý trellis, giai đoạn traceback cần khôi phục chuỗi dữ liệu gồm 8 bit.

- Thuật toán Viterbi gồm các bước lý thuyết chính: tính branch metric, cập nhật path metric, chọn survivor path và thực hiện traceback. Các bước này sẽ là nền tảng để xây dựng kiến trúc phần cứng của bộ giải mã Viterbi ở chương tiếp theo.

CHƯƠNG 2. THIẾT KẾ VITERBI DECODER

2.1. Top module (viterbi_decoder)

2.1.1. Sơ đồ khối



Hình 2.1. Sơ đồ khối Top module

2.1.2. Bảng dữ liệu đầu ra đầu vào

Bảng 2.1. Bảng dữ liệu I/O của Top module

Tên tín hiệu	**I/O**	**Số bit**	**Kiểu dữ liệu**	**Mô tả**
clk	Input	1	wire	Xung đồng hồ hệ thống
rst_n	Input	1	wire	Reset bất đồng bộ tích cực mức thấp
en	Input	1	wire	Tín hiệu cho phép hệ thống bắt đầu giải mã
encoded_in	Input	16	wire	Chuỗi dữ liệu mã hóa đầu vào gồm 16 bit
decoded_out	Output	8	wire/reg	Chuỗi dữ liệu sau giải mã gồm 8 bit
done	Output	1	wire/reg	Báo quá trình giải mã hoàn tất

2.1.3. Chức năng

Khối viterbi_top là khối cấp cao nhất của bộ giải mã Viterbi, có nhiệm vụ tích hợp các khối con trong hệ thống và tổ chức luồng xử lý dữ liệu từ đầu vào mã hóa đến đầu ra giải mã. Khối này nhận chuỗi dữ liệu mã hóa encoded_in[15:0], tín hiệu xung đồng hồ clk, tín hiệu reset tích cực mức thấp rst_n và tín hiệu cho phép hoạt động en. Sau khi quá trình giải mã hoàn tất, khối xuất chuỗi dữ liệu đã giải mã decoded_out[7:0] cùng tín hiệu done để báo kết quả đầu ra đã hợp lệ.

Trong thiết kế này, viterbi_top không thực hiện toàn bộ thuật toán Viterbi trong một mạch đơn lẻ mà đóng vai trò kết nối và điều phối các khối chức năng

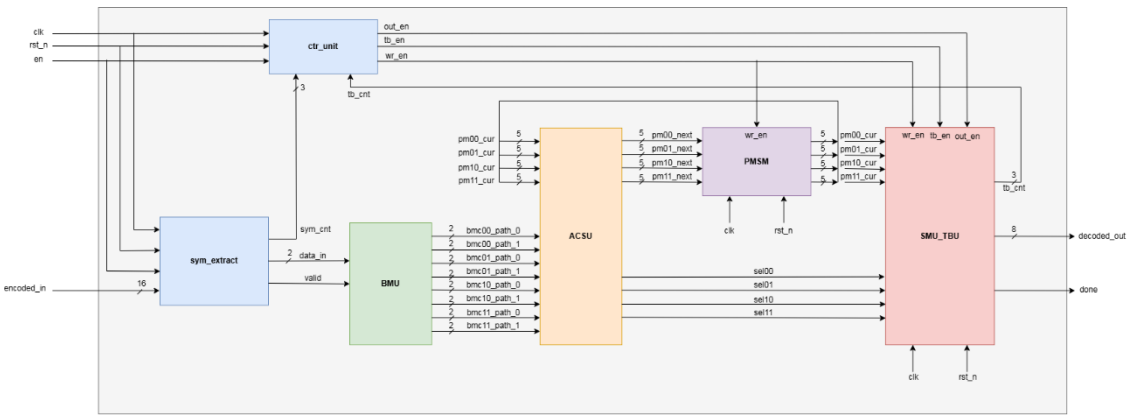
chuyên biệt. Chuỗi mã hóa 16 bit trước tiên được đưa vào khối sym_extract để tách lần lượt thành các symbol 2 bit. Mỗi symbol hợp lệ sau đó được truyền đến khối bmu để tính các branch metric tương ứng với các nhánh trong sơ đồ trellis. Các giá trị branch metric này tiếp tục được đưa sang khối acsu, nơi thực hiện quá trình cộng, so sánh và lựa chọn đường đi có metric nhỏ hơn cho từng trạng thái.

Khối pmsm được sử dụng để lưu trữ path metric hiện tại của bốn trạng thái trellis. Các path metric mới do acsu tính toán sẽ được ghi trở lại pmsm khi tín hiệu điều khiển ghi được kích hoạt. Nhờ đó, hệ thống có thể cập nhật chi phí đường đi tích lũy qua từng symbol đầu vào. Đồng thời, các decision bit do acsu tạo ra được đưa đến khối smu_tbu để lưu lại thông tin survivor path, phục vụ cho giai đoạn truy vết ngược.

Sau khi toàn bộ 8 symbol đầu vào đã được xử lý, khối smu_tbu thực hiện traceback dựa trên các decision bit đã lưu và các path metric hiện tại. Quá trình này cho phép khôi phục lại chuỗi dữ liệu ban đầu gồm 8 bit. Khi traceback hoàn tất, dữ liệu giải mã được đưa ra cổng decoded_out[7:0] và tín hiệu done được kích hoạt để báo hiệu quá trình giải mã kết thúc.

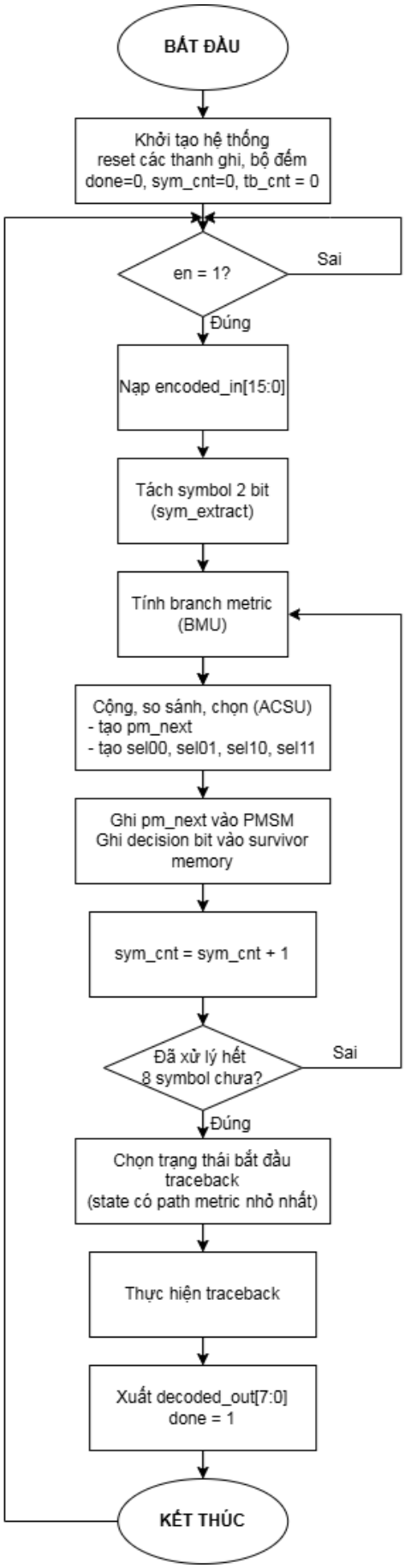
Bên cạnh đường dữ liệu chính, khối viterbi_top còn tích hợp khối ctrl_unit để tạo các tín hiệu điều khiển nội bộ gồm wr_en, tb_en và out_en. Các tín hiệu này lần lượt điều khiển quá trình ghi path metric và decision bit, kích hoạt traceback và cho phép xuất dữ liệu đầu ra. Nhờ sự phối hợp giữa đường dữ liệu và đường điều khiển, khối top đảm bảo các khối con hoạt động đúng trình tự: tách symbol, tính branch metric, cập nhật path metric, lưu survivor path, traceback và xuất kết quả giải mã.

2.1.4. Sơ đồ nối dây bên trong



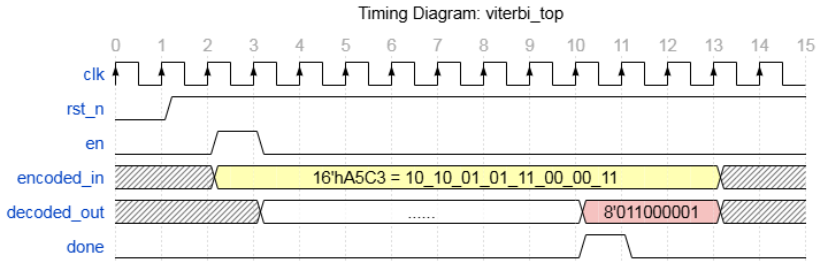
Hình 2.2. Sơ đồ nối dây bên trong Top module

2.1.5. Lưu đồ thuật toán tổng quát viterbi decoder



Hình 2.3. Lưu đồ thuật toán tổng quát Viterbi decoder

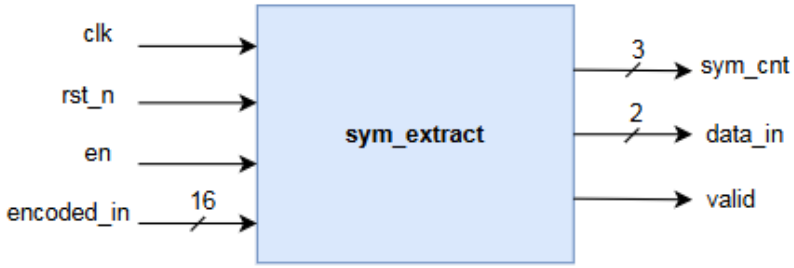
2.1.6. Timing diagram tổng quát viterbi decoder



Hình 2.4.Timing diagram tổng quát Viterbi decoder

2.2. Khối Symbol Extractor (sym_extract)

2.2.1. Sơ đồ khối



Hình 2.5. Sơ đồ khối của khối sym_extract

2.2.2. Bảng dữ liệu đầu ra đầu vào

Bảng 2.2. Bảng dữ liệu I/O của khối sym_extract

Tên tín hiệu	I/O	Số bit	Kiểu dữ liệu	Mô tả
clk	Input	1	wire	Xung đồng hồ hệ thống
rst_n	Input	1	wire	Reset bất đồng bộ tích cực mức thấp
en	Input	1	wire	Tín hiệu cho phép hệ thống bắt đầu giải mã
encoded_in	Input	16	wire	Chuỗi dữ liệu mã hóa đầu vào gồm 16 bit
data_in	Output	2	wire/reg	Cặp bit đầu ra đưa sang BMU
sym_cnt	Output	3	wire/reg	Bộ đếm số symbol đã xuất
valid	Output	1	wire/reg	Cờ báo data_in hợp lệ

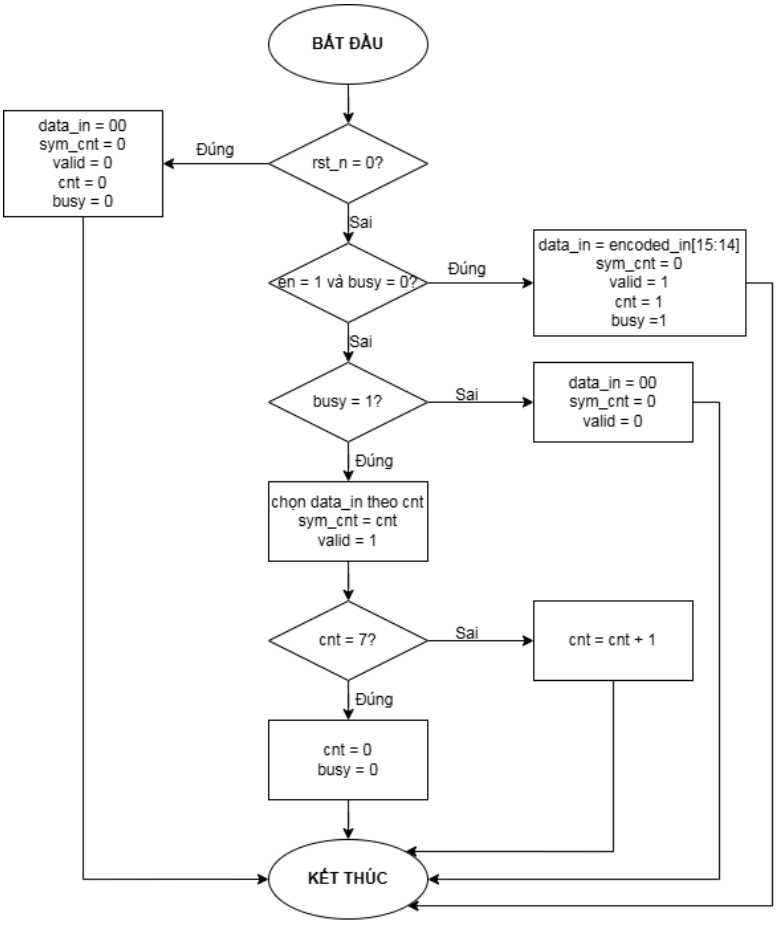
2.2.3. Chức năng

Khối sym_extract có nhiệm vụ tách chuỗi mã hóa encoded_in[15:0] thành 8 symbol 2 bit để đưa sang khối BMU. Các symbol được xuất tuần tự từ bit có trọng số cao đến bit có trọng số thấp, bắt đầu từ encoded_in[15:14] và kết thúc tại encoded_in[1:0].

Khi en được kích hoạt, khối xuất symbol đầu tiên ra data_in[1:0], đặt sym_cnt = 0, đưa valid = 1 và bật tín hiệu nội bộ busy để bắt đầu quá trình tách dữ liệu. Bộ đếm nội bộ cnt được dùng để xác định vị trí symbol tiếp theo cần xuất. Trong các chu kỳ sau, khi busy = 1, khối lần lượt đưa các cặp bit còn lại của encoded_in ra data_in, đồng thời cập nhật sym_cnt theo giá trị cnt.

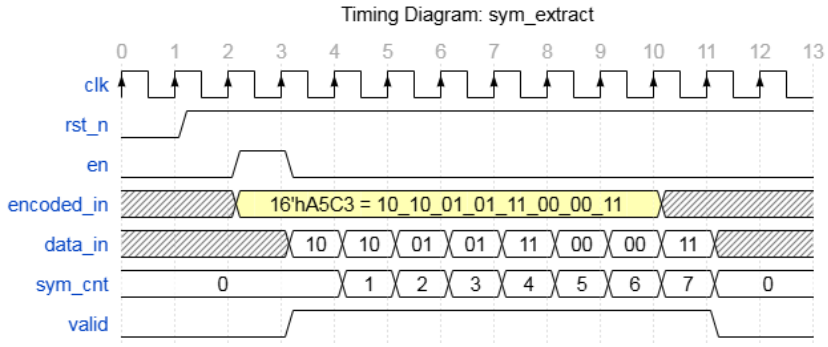
Tín hiệu valid cho biết dữ liệu tại data_in đang hợp lệ. Sau khi xuất đủ 8 symbol, tức là khi cnt đạt giá trị 7, khối xóa busy, đưa cnt về 0 và kết thúc quá trình tách dữ liệu. Khi không hoạt động, data_in, sym_cnt và valid được đưa về giá trị mặc định. Như vậy, sym_extract đóng vai trò chuyển dữ liệu mã hóa 16 bit dạng song song thành luồng symbol 2 bit tuần tự cho các bước xử lý tiếp theo.

2.2.4. Lưu đồ thuật toán



Hình 2.6. Lưu đồ thuật toán của khối sym_extract

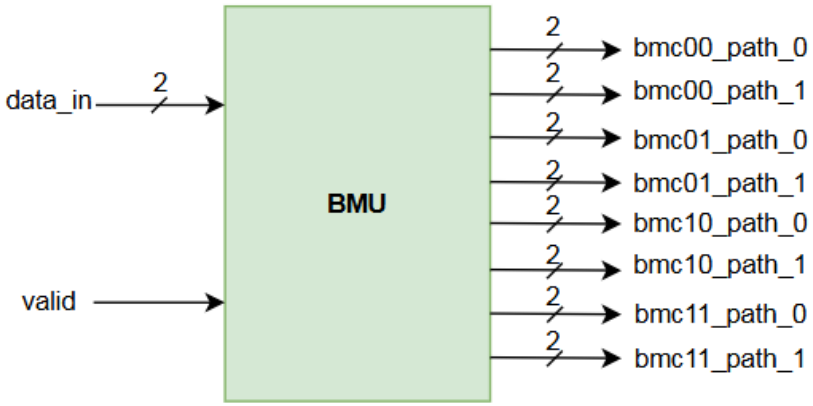
2.2.5. Timing diagram



Hình 2.7. Timing diagram của khối sym_extract

2.3. Khối Branch Metric Unit (bmu)

2.3.1. Sơ đồ khối



Hình 2.8. Sơ đồ khối của khối bmu

2.3.2. Bảng dữ liệu đầu ra đầu vào

Bảng 2.3. Bảng dữ liệu I/O của khối bmu

Tên tín hiệu	I/O	Số bit	Kiểu dữ liệu	Mô tả
data_in	Input	2	wire	Cặp dữ liệu đầu vào cần giải mã
valid	Input	1	wire	Cờ báo data_in hợp lệ
bmc00_path_0	Output	2	wire	Branch metric của nhánh đi từ trạng thái 00 với input bit 0
bmc00_path_1	Output	2	wire	Branch metric của nhánh đi từ trạng thái 00 với input bit 1
bmc01_path_0	Output	2	wire	Branch metric của nhánh đi từ trạng thái 01 với input bit 0
bmc01_path_1	Output	2	wire	Branch metric của nhánh đi từ trạng thái 01 với input bit 1

bmc10_path_0	Output	2	wire	Branch metric của nhánh đi từ trạng thái 10 với input bit 0
bmc10_path_1	Output	2	wire	Branch metric của nhánh đi từ trạng thái 10 với input bit 1
bmc11_path_0	Output	2	wire	Branch metric của nhánh đi từ trạng thái 11 với input bit 0
bmc11_path_1	Output	2	wire	Branch metric của nhánh đi từ trạng thái 11 với input bit 1

2.3.3. Chức năng

Khối bmu có nhiệm vụ tính branch metric cho các nhánh trong sơ đồ trellis của bộ giải mã Viterbi. Đầu vào của khối là symbol 2 bit data_in[1:0] từ khối sym_extract và tín hiệu valid báo dữ liệu đầu vào hợp lệ. Khi valid = 1, khối so sánh data_in với các mẫu bit kỳ vọng trên trellis để tạo ra các giá trị branch metric đưa sang khối acsu.

Trong thiết kế này, branch metric được tính bằng khoảng cách Hamming giữa data_in và bốn mẫu tham chiếu 00, 01, 10, 11. Các khoảng cách này được biểu diễn bằng các tín hiệu nội bộ dist_00, dist_01, dist_10 và dist_11. Mỗi tín hiệu có độ rộng 2 bit, tương ứng với số bit sai khác giữa symbol đầu vào và mẫu tham chiếu.

Sau khi tính các khoảng cách Hamming cơ bản, khối bmu ánh xạ chúng thành tám đầu ra branch metric gồm bmc00_path_0, bmc00_path_1, bmc01_path_0, bmc01_path_1, bmc10_path_0, bmc10_path_1, bmc11_path_0 và bmc11_path_1. Các tín hiệu này tương ứng với hai nhánh xuất phát từ mỗi trạng thái 00, 01, 10 và 11. Khi valid = 0, toàn bộ branch metric được đưa về 2'b00 để tránh truyền dữ liệu không hợp lệ sang khối phía sau.

BMU đóng vai trò tính chi phí nhánh cho từng symbol đầu vào. Kết quả của khối được sử dụng bởi acsu để cập nhật path metric và lựa chọn survivor path trong quá trình giải mã.

2.3.4. Lưu đồ thuật toán



Hình 2.9. Lưu đồ thuật toán của khối bmu

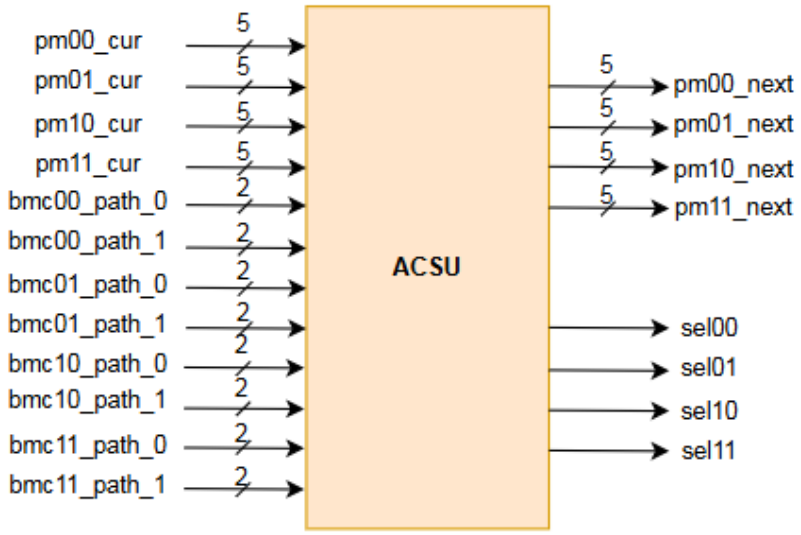
2.3.5. Timing diagram



Hình 2.10. Timing diagram của khối bmu

2.4. Khối Add-Compare-Select Unit (acsu)

2.4.1. Sơ đồ khối



Hình 2.11. Sơ đồ khối của khối acsu

2.4.2. Bảng dữ liệu đầu ra đầu vào

Bảng 2.4. Bảng dữ liệu I/O của khối acsu

Tên tín hiệu	I/O	Số bit	Kiểu dữ liệu	Mô tả
bmc00_path_0	Input	2	wire	Branch metric của nhánh đi từ trạng thái 00 với input bit 0
bmc00_path_1	Input	2	wire	Branch metric của nhánh đi từ trạng thái 00 với input bit 1
bmc01_path_0	Input	2	wire	Branch metric của nhánh đi từ trạng thái 01 với input bit 0
bmc01_path_1	Input	2	wire	Branch metric của nhánh đi từ trạng thái 01 với input bit 1
bmc10_path_0	Input	2	wire	Branch metric của nhánh đi từ trạng thái 10 với input bit 0
bmc10_path_1	Input	2	wire	Branch metric của nhánh đi từ trạng thái 10 với input bit 1
bmc11_path_0	Input	2	wire	Branch metric của nhánh đi từ trạng thái 11 với input bit 0
bmc11_path_1	Input	2	wire	Branch metric của nhánh đi từ trạng thái 11 với input bit 1
pm00_cur	Input	5	wire	Path metric cũ của trạng thái 00

pm01_cur	Input	5	wire	Path metric cũ của trạng thái 01
pm10_cur	Input	5	wire	Path metric cũ của trạng thái 10
pm11_cur	Input	5	wire	Path metric cũ của trạng thái 11
pm00_next	Output	5	wire	Path metric mới của trạng thái 00
pm01_next	Output	5	wire	Path metric mới của trạng thái 01
pm10_next	Output	5	wire	Path metric mới của trạng thái 10
pm11_next	Output	5	wire	Path metric mới của trạng thái 11
sel00	Output	1	wire	Decision bit của trạng thái 00
sel01	Output	1	wire	Decision bit của trạng thái 01
sel10	Output	1	wire	Decision bit của trạng thái 10
sel11	Output	1	wire	Decision bit của trạng thái 11

2.4.3. Chức năng

Khối acsu thực hiện chức năng cộng, so sánh và lựa chọn đường sống sót trong thuật toán Viterbi. Khối nhận các branch metric từ bmu và các path metric hiện tại từ pmsm, sau đó tính toán path metric mới cho bốn trạng thái trellis gồm 00, 01, 10 và 11. Kết quả đầu ra của khối gồm các path metric mới pm00_next, pm01_next, pm10_next, pm11_next và các bit lựa chọn sel00, sel01, sel10, sel11.

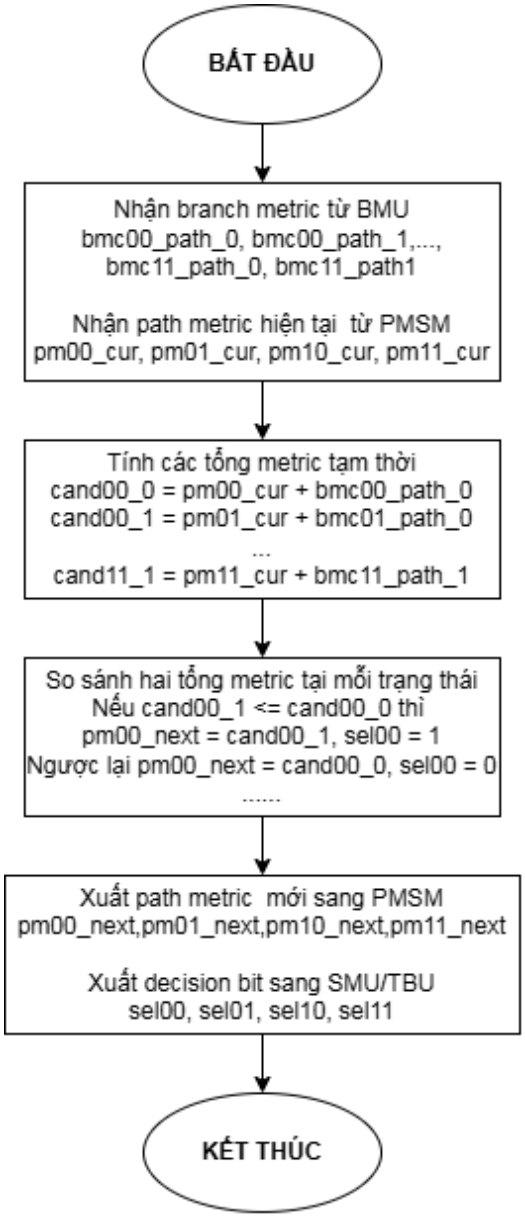
Với mỗi trạng thái kế tiếp, có hai đường đi khả dĩ hội tụ vào trạng thái đó. Khối acsu cộng path metric hiện tại của trạng thái trước với branch metric tương ứng để tạo ra các tổng metric tạm thời. Các tín hiệu nội bộ cand00_0, cand00_1, cand01_0, cand01_1, cand10_0, cand10_1, cand11_0 và cand11_1 được dùng để biểu diễn các tổng metric tạm thời này. Mỗi tín hiệu có độ rộng 6 bit nhằm hạn chế mất dữ liệu trong quá trình cộng trước khi đưa vào khối so sánh.

Sau khi tạo hai tổng metric tạm thời cho từng trạng thái kế tiếp, khối tiến hành so sánh và chọn giá trị nhỏ hơn làm path metric mới. Cụ thể, pm00_next được chọn từ hai đường đi hội tụ vào trạng thái 00, pm01_next được chọn từ hai đường đi hội tụ vào trạng thái 01, pm10_next được chọn từ hai đường đi hội tụ vào trạng thái 10 và pm11_next được chọn từ hai đường đi hội tụ vào trạng thái 11. Các tín

hiệu sel00, sel01, sel10 và sel11 cho biết nhánh được chọn làm survivor path của từng trạng thái. Trong trường hợp hai tổng metric bằng nhau, logic so sánh sử dụng điều kiện <=, do đó nhánh thứ hai được ưu tiên chọn theo mô tả RTL hiện tại.

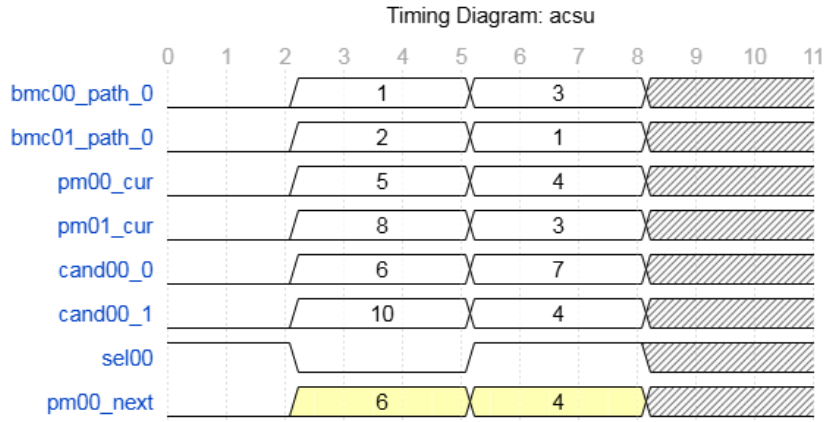
Khối acsu là mạch tổ hợp, không sử dụng tín hiệu clock và không lưu trạng thái qua chu kỳ. Path metric mới do acsu tạo ra được ghi vào pmsm khi có tín hiệu cho phép ghi từ khối điều khiển. Đồng thời, các bit lựa chọn được đưa sang smu_tbu để lưu lại thông tin survivor path phục vụ cho quá trình traceback.

2.4.4. Lưu đồ thuật toán



Hình 2.12. Lưu đồ thuật toán của khối acsu

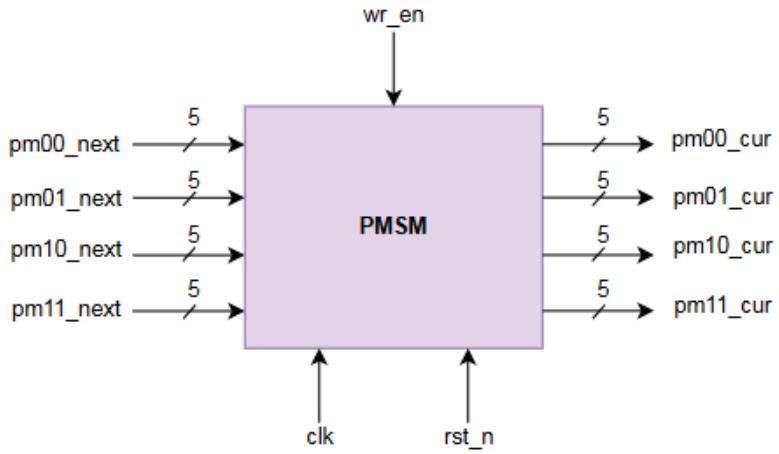
2.4.5. Timing diagram



Hình 2.13. Timing diagram của khối acsu

2.5. Khối Path Metric Store Memory (pmsm)

2.5.1. Sơ đồ khối



Hình 2.14. Sơ đồ khối của khối pmsm

2.5.2. Bảng dữ liệu đầu ra đầu vào

Bảng 2.5. Bảng dữ liệu I/O của khối pmsm

Tên tín hiệu	I/O	Số bit	Kiểu dữ liệu	Mô tả
clk	Input	1	wire	Xung đồng hồ hệ thống
rst_n	Input	1	wire	Reset bất đồng bộ tích cực mức thấp
wr_en	Input	1	wire	Cho phép ghi path metric mới vào bộ nhớ
pm00_next	Input	5	wire	Path metric mới của trạng thái 00
pm01_next	Input	5	wire	Path metric mới của trạng thái 01

pm10_next	Input	5	wire	Path metric mới của trạng thái 10
pm11_next	Input	5	wire	Path metric mới của trạng thái 11
pm00_cur	Output	5	wire/reg	Path metric hiện tại của trạng thái 00
pm01_cur	Output	5	wire/reg	Path metric hiện tại của trạng thái 01
pm10_cur	Output	5	wire/reg	Path metric hiện tại của trạng thái 10
pm11_cur	Output	5	wire/reg	Path metric hiện tại của trạng thái 11

2.5.3. Chức năng

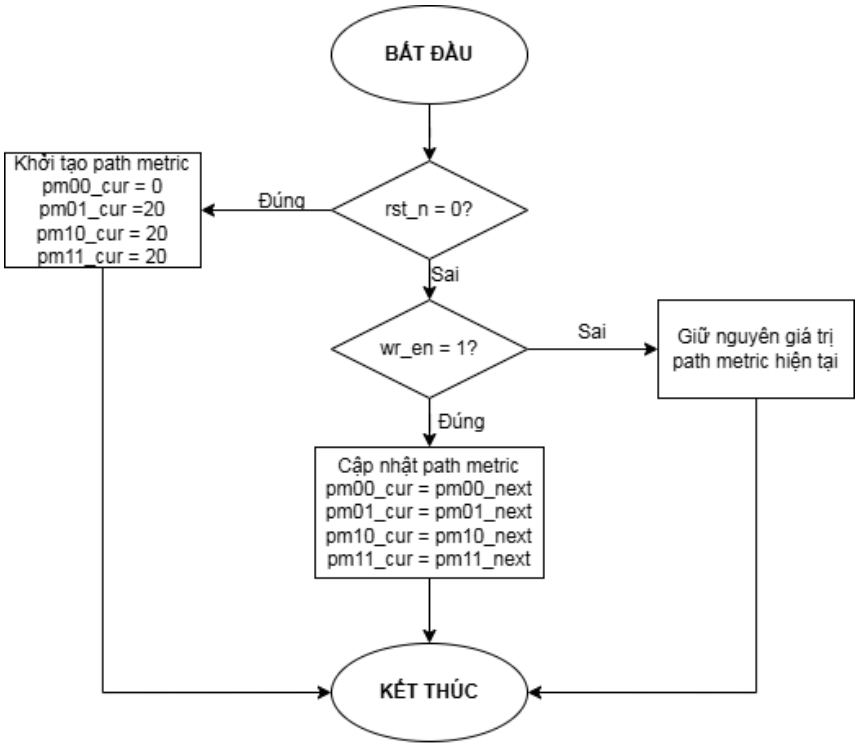
Khối pmsm có nhiệm vụ lưu trữ path metric hiện tại của bốn trạng thái trong trellis, gồm 00, 01, 10 và 11. Các giá trị path metric này được sử dụng làm đầu vào cho khối acsu ở bước xử lý tiếp theo. Đầu vào của khối là các path metric mới pm00_next, pm01_next, pm10_next, pm11_next do acsu tạo ra, còn đầu ra là các path metric hiện tại pm00_cur, pm01_cur, pm10_cur, pm11_cur.

Khi reset, khối khởi tạo pm00_cur = 0, trong khi các trạng thái còn lại pm01_cur, pm10_cur và pm11_cur được đặt bằng 20. Cách khởi tạo này phản ánh giả thiết ban đầu của thuật toán Viterbi: đường đi xuất phát từ trạng thái 00 có metric nhỏ nhất, còn các trạng thái khác được gán giá trị lớn để chưa được ưu tiên trong bước tính đầu tiên.

Trong quá trình hoạt động, nếu tín hiệu wr_en = 1, khối ghi các path metric mới từ acsu vào các thanh ghi tương ứng. Khi đó, pm00_next, pm01_next, pm10_next, pm11_next được cập nhật thành pm00_cur, pm01_cur, pm10_cur, pm11_cur. Nếu wr_en = 0, khối giữ nguyên các giá trị path metric hiện tại, không thực hiện cập nhật.

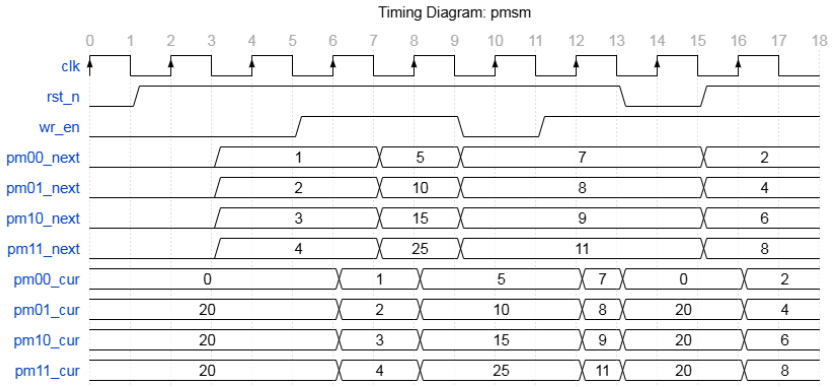
Như vậy, pmsm đóng vai trò là bộ nhớ path metric của bộ giải mã Viterbi. Khối này tạo vòng phản hồi giữa acsu và chính các giá trị path metric đã tích lũy, giúp hệ thống cập nhật chi phí đường đi qua từng symbol đầu vào.

2.5.4. Lưu đồ thuật toán



Hình 2.15. Lưu đồ thuật toán của khối pmsm

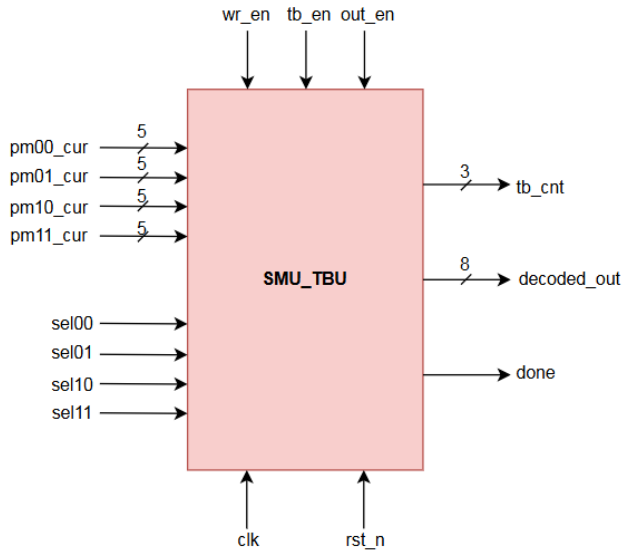
2.5.5. Timing diagram



Hình 2.16. Timing diagram của khối pmsm

2.6. Khối Survivor Memory and Traceback Unit (smu_tbu)

2.6.1. Sơ đồ khối



Hình 2.17. Sơ đồ khối của khối smu_tbu

2.6.2. Bảng dữ liệu đầu ra đầu vào

Bảng 2.6. Bảng dữ liệu I/O của khối smu_tbu

Tên tín hiệu	**I/O**	**Số bit**	**Kiểu dữ liệu**	**Mô tả**
clk	Input	1	wire	Xung đồng hồ hệ thống
rst_n	Input	1	wire	Reset bất đồng bộ tích cực mức thấp
wr_en	Input	1	wire	Cho phép ghi decision bit vào bộ nhớ survivor
tb_en	Input	1	wire	Cho phép bắt đầu quá trình traceback
out_en	Input	1	wire	Cho phép xuất dữ liệu giải mã
sel00	Input	1	wire	Decision bit của trạng thái 00
sel01	Input	1	wire	Decision bit của trạng thái 01
sel10	Input	1	wire	Decision bit của trạng thái 10
sel11	Input	1	wire	Decision bit của trạng thái 11
pm00_cur	Input	5	wire	Path metric hiện tại của trạng thái 00
pm01_cur	Input	5	wire	Path metric hiện tại của trạng thái 01
pm10_cur	Input	5	wire	Path metric hiện tại của trạng thái 10

pm11_cur	Input	5	wire	Path metric hiện tại của trạng thái 11
decoded_out	Output	8	wire/reg	Chuỗi dữ liệu giải mã đầu ra
tb_cnt	Output	3	wire/reg	Bộ đếm số bước traceback
done	Output	1	wire/reg	Báo quá trình traceback hoàn tất

2.6.3. Chức năng

Khối smu_tbu thực hiện hai chức năng chính trong bộ giải mã Viterbi: lưu thông tin survivor path và thực hiện traceback để khôi phục dữ liệu đầu ra. Khối nhận các bit lựa chọn sel00, sel01, sel10, sel11 từ acsu, các path metric hiện tại pm00_cur, pm01_cur, pm10_cur, pm11_cur từ pmsm, cùng các tín hiệu điều khiển wr_en, tb_en và out_en từ ctrl_unit.

Trong giai đoạn ghi, khi wr_en = 1, các decision bit từ acsu được lưu vào bốn thanh ghi nội bộ surv00, surv01, surv10 và surv11. Mỗi thanh ghi có độ rộng 8 bit, tương ứng với 8 bước xử lý symbol của chuỗi đầu vào. Ở mỗi chu kỳ ghi, decision bit mới được dịch vào bit thấp nhất của từng thanh ghi survivor, còn các giá trị cũ được dịch sang vị trí kế tiếp. Nhờ đó, khối lưu lại lịch sử lựa chọn nhánh sống sót của bốn trạng thái trellis 00, 01, 10 và 11.

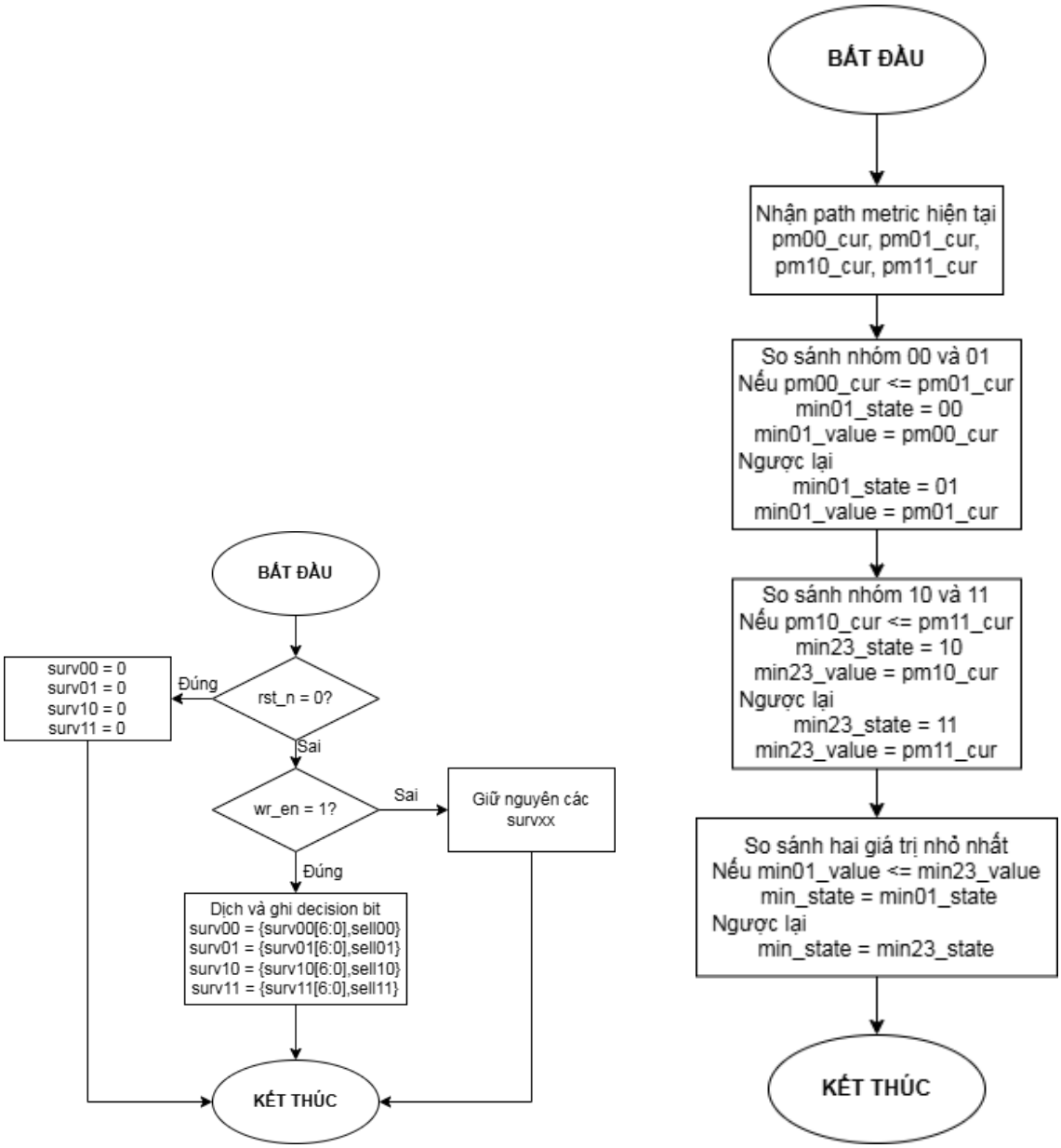
Trước khi traceback, khối chọn trạng thái bắt đầu dựa trên path metric nhỏ nhất trong bốn trạng thái hiện tại. Việc so sánh được thực hiện theo hai tầng: trước tiên so sánh pm00_cur với pm01_cur để tạo min01_state và min01_value, đồng thời so sánh pm10_cur với pm11_cur để tạo min23_state và min23_value. Sau đó, hai giá trị nhỏ nhất trung gian tiếp tục được so sánh để xác định min_state. Trạng thái này được nạp vào thanh ghi nội bộ cur_state khi khối ở trạng thái chờ.

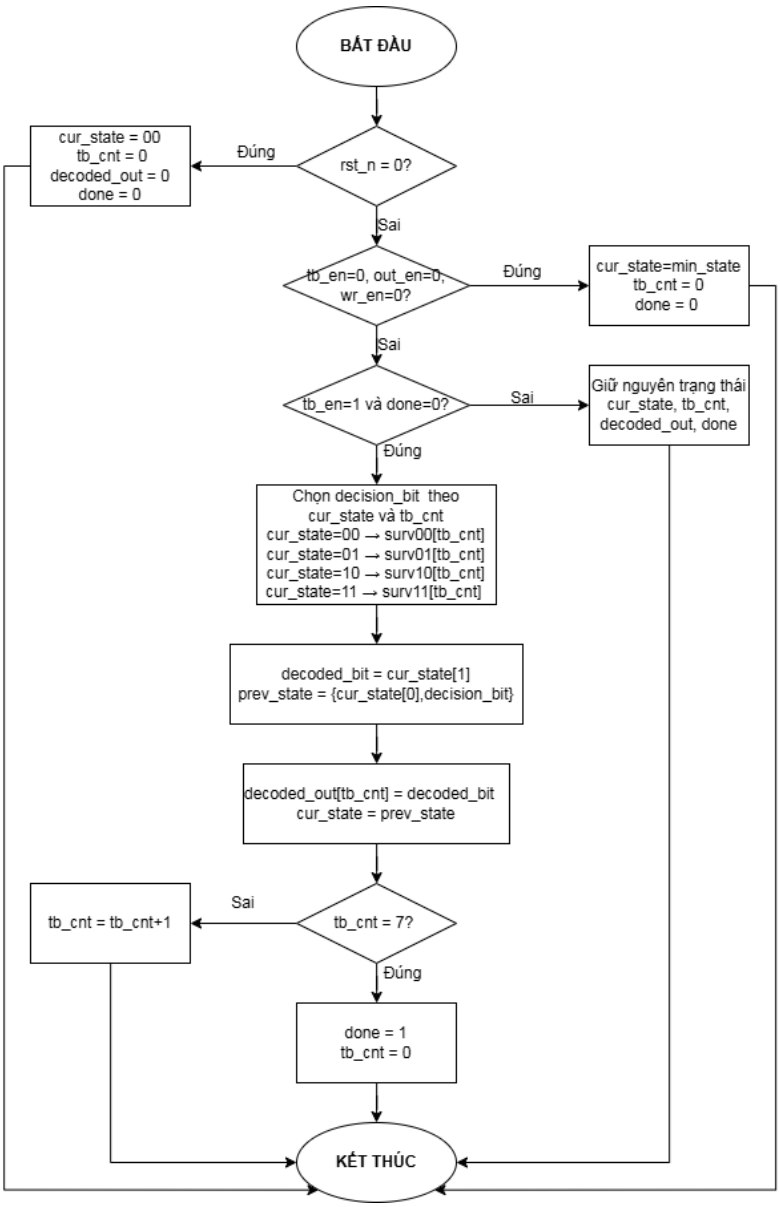
Khi tb_en = 1, khối bắt đầu quá trình traceback. Tại mỗi bước traceback, khối sử dụng cur_state và bộ đếm tb_cnt để chọn decision bit tương ứng từ một trong bốn thanh ghi surv00, surv01, surv10 hoặc surv11. Decision bit này được dùng để xác định trạng thái trước đó thông qua tín hiệu nội bộ prev_state. Đồng thời, bit giải mã tại bước hiện tại được lấy từ cur_state[1] và ghi vào decoded_out[tb_cnt].

Bộ đếm tb_cnt dùng để xác định vị trí bit đang được truy vết và ghi vào decoded_out. Quá trình traceback tiếp tục cho đến khi tb_cnt đạt giá trị 7, tương

ứng với việc đã khôi phục đủ 8 bit dữ liệu. Khi đó, tín hiệu done được đưa lên mức 1 để báo quá trình giải mã hoàn tất, đồng thời tb_cnt được đưa về 0. Như vậy, smu_tbu là khối cuối cùng trong đường xử lý dữ liệu, có nhiệm vụ chuyển thông tin survivor path đã lưu thành chuỗi dữ liệu giải mã decoded_out[7:0].

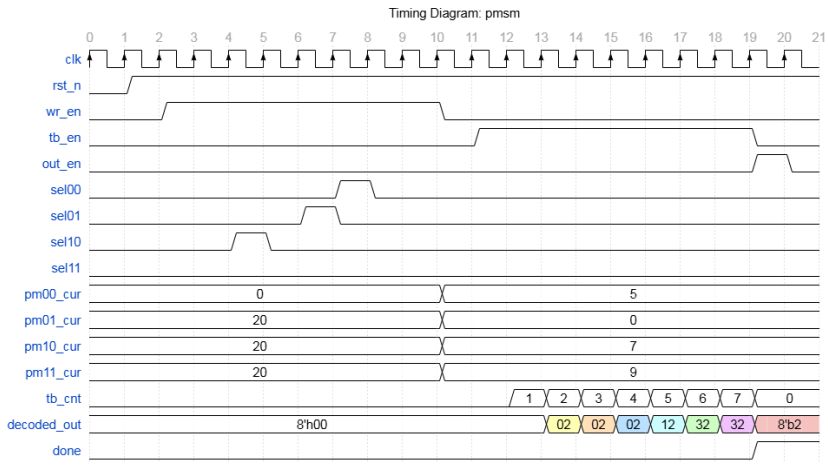
2.6.4. Lưu đồ thuật toán





Hình 2.18. (a) Lưu đồ ghi decision bit vào survivor memory
(b) Lưu đồ chọn min_state cho quá trình traceback
(c) Lưu đồ thực hiện traceback trong khối smu_tbu

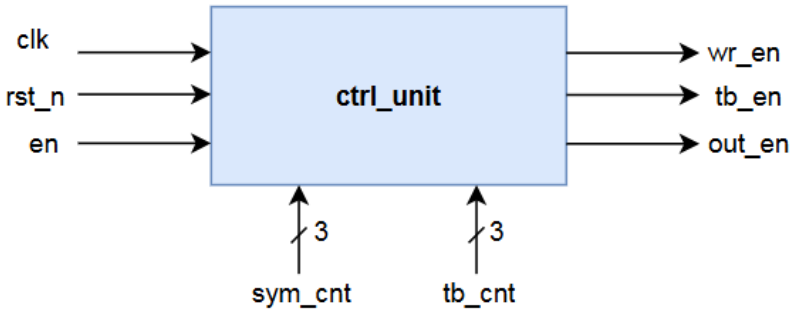
2.6.5. Timing diagram



Hình 2.19. Timing diagram của khối smu_tbu

2.7. Khối Control Unit (ctr_unit)

2.7.1. Sơ đồ khối



Hình 2.20. Sơ đồ khối của khối ctrl_unit

2.7.2. Bảng dữ liệu đầu ra đầu vào

Bảng 2.7. Bảng dữ liệu I/O của khối ctrl_unit

Tên tín hiệu	I/O	Số bit	Kiểu dữ liệu	Mô tả
clk	Input	1	wire	Xung đồng hồ hệ thống
rst_n	Input	1	wire	Reset bất đồng bộ tích cực mức thấp
en	Input	1	wire	Tín hiệu cho phép bắt đầu giải mã
sym_cnt	Input	3	wire	Bộ đếm số symbol đã xử lý từ khối sym_extractor
tb_cnt	Input	3	wire	Bộ đếm số bước traceback
wr_en	Output	1	wire/reg	Cho phép ghi dữ liệu vào các khối bộ nhớ
tb_en	Output	1	wire/reg	Cho phép bắt đầu traceback
out_en	Output	1	wire/reg	Cho phép xuất dữ liệu giải mã

2.7.3. Chức năng

Khối ctrl_unit đóng vai trò là bộ điều khiển trung tâm của bộ giải mã Viterbi. Khối này tạo ra các tín hiệu điều khiển wr_en, tb_en và out_en để điều phối hoạt động của các khối lưu path metric, lưu survivor path và traceback. Đầu vào của khối gồm tín hiệu cho phép bắt đầu en, bộ đếm symbol sym_cnt, bộ đếm traceback tb_cnt và tín hiệu hoàn tất done từ khối smu_tbu.

Trong thiết kế hiện tại, ctrl_unit được xây dựng dưới dạng máy trạng thái hữu hạn gồm năm trạng thái: IDLE, PROCESS, WAIT, TB và OUT. Ở trạng thái

IDLE, toàn bộ tín hiệu điều khiển được đưa về 0. Khi en = 1, bộ điều khiển chuyển sang trạng thái PROCESS để bắt đầu quá trình xử lý symbol đầu vào.

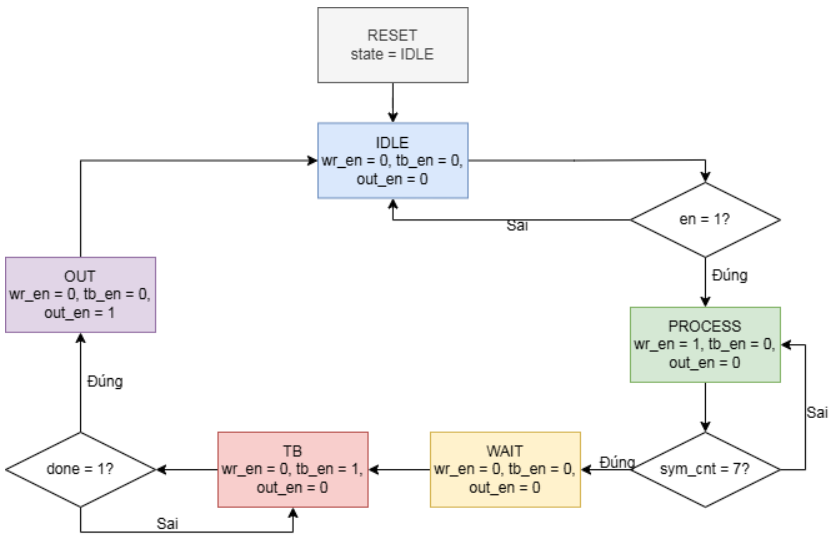
Tại trạng thái PROCESS, tín hiệu wr_en được kích hoạt để cho phép khối pmsm cập nhật path metric và khối smu_tbu ghi các decision bit vào survivor memory. Trạng thái này được duy trì trong quá trình khối sym_extract xuất các symbol đầu vào. Khi sym_cnt = 7, tức là đã xử lý đến symbol cuối cùng của chuỗi 16 bit, bộ điều khiển chuyển sang trạng thái WAIT.

Trạng thái WAIT là trạng thái trung gian giữa quá trình xử lý symbol và quá trình traceback. Trong trạng thái này, các tín hiệu wr_en, tb_en và out_en đều bằng 0. Sau một chu kỳ, bộ điều khiển chuyển sang trạng thái TB để kích hoạt quá trình truy vết ngược.

Ở trạng thái TB, tín hiệu tb_en được đưa lên mức 1, cho phép khối smu_tbu thực hiện traceback dựa trên các survivor bit đã lưu. Bộ điều khiển giữ trạng thái này cho đến khi nhận được tín hiệu done = 1 từ smu_tbu, báo rằng quá trình traceback đã hoàn tất. Khi đó, FSM chuyển sang trạng thái OUT.

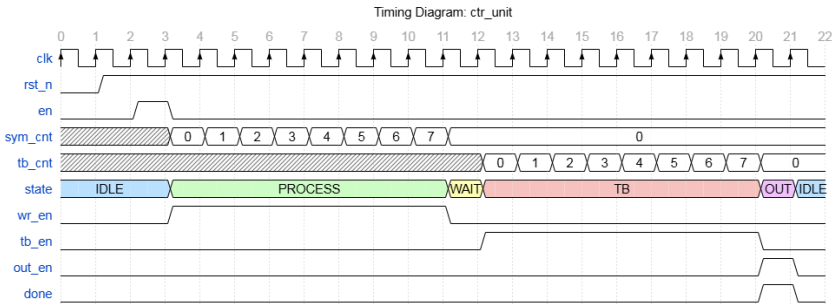
Trong trạng thái OUT, tín hiệu out_en được kích hoạt trong một chu kỳ để giữ và xác nhận dữ liệu đầu ra sau traceback. Sau trạng thái này, bộ điều khiển quay trở lại IDLE, sẵn sàng cho lần giải mã tiếp theo.

2.7.4. Lưu đồ thuật toán



Hình 2.21. Lưu đồ thuật toán của khối ctrl_unit

2.7.5. Timing diagram



Hình 2.22. Timing diagram của khối ctrl_unit

CHƯƠNG 3. KẾT QUẢ MÔ PHỎNG

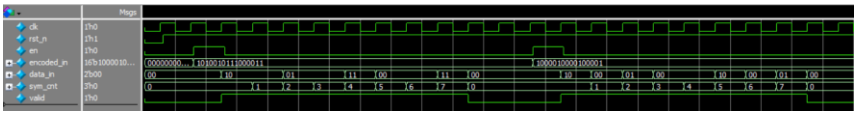
3.1. Khối sym_extract

Bảng 3.1. Testcase kiểm chứng khối sym_extract

Tên testcase	Mục đích	Điều kiện kiểm thử	Kết quả mong đợi
TC1: Reset	Kiểm tra reset	rst_n = 0, en = 0	data_in = 00, sym_cnt = 0, valid = 0
TC2: Start block 1	Kiểm tra xung bắt đầu	encoded_in = 16'hA5C3, en = 1 trong 1 chu kỳ	Bắt đầu tách từ MSB, data_in = 10, sym_cnt = 0, valid = 1
TC3: Extract block 1	Kiểm tra tách 8 symbol	encoded_in = 16'hA5C3	data_in = 10, 10, 01, 01, 11, 00, 00, 11; sym_cnt = 0→7
TC4: End block 1	Kiểm tra tự dừng	Sau khi xuất đủ 8 symbol block 1	data_in = 00, sym_cnt = 0, valid = 0
TC5: Start block 2	Kiểm tra khởi động lại	encoded_in = 16'h8421, en = 1 trong 1 chu kỳ	Bắt đầu block mới, data_in = 10, sym_cnt = 0, valid = 1
TC6: Extract block 2	Kiểm tra tách block mới	encoded_in = 16'h8421	data_in = 10, 00, 01, 00, 00, 10, 00, 01; sym_cnt = 0→7
TC7: End block 2	Kiểm tra tự dừng	Sau khi xuất đủ 8 symbol block 2	data_in = 00, sym_cnt = 0, valid = 0

```
# ===== START SYM_EXTRACT TESTBENCH =====
# --- TESTCASE 1: RESET ---
# --- TESTCASE 2: NORMAL OPERATION ---
# [PASS]  @ 26000 ns: MATCH -> data=10, cnt=0, valid=1
# [PASS]  @ 36000 ns: MATCH -> data=10, cnt=1, valid=1
# [PASS]  @ 46000 ns: MATCH -> data=01, cnt=2, valid=1
# [PASS]  @ 56000 ns: MATCH -> data=01, cnt=3, valid=1
# [PASS]  @ 66000 ns: MATCH -> data=11, cnt=4, valid=1
# [PASS]  @ 76000 ns: MATCH -> data=00, cnt=5, valid=1
# [PASS]  @ 86000 ns: MATCH -> data=00, cnt=6, valid=1
# [PASS]  @ 96000 ns: MATCH -> data=11, cnt=7, valid=1
# [PASS]  @ 106000 ns: MATCH -> data=00, cnt=0, valid=0
# [PASS]  @ 116000 ns: MATCH -> data=00, cnt=0, valid=0
# --- TESTCASE 3: RESTART NEW BLOCK ---
# [PASS]  @ 136000 ns: MATCH -> data=10, cnt=0, valid=1
# [PASS]  @ 146000 ns: MATCH -> data=00, cnt=1, valid=1
# [PASS]  @ 156000 ns: MATCH -> data=01, cnt=2, valid=1
# [PASS]  @ 166000 ns: MATCH -> data=00, cnt=3, valid=1
# [PASS]  @ 176000 ns: MATCH -> data=00, cnt=4, valid=1
# [PASS]  @ 186000 ns: MATCH -> data=10, cnt=5, valid=1
# [PASS]  @ 196000 ns: MATCH -> data=00, cnt=6, valid=1
# [PASS]  @ 206000 ns: MATCH -> data=01, cnt=7, valid=1
# [PASS]  @ 216000 ns: MATCH -> data=00, cnt=0, valid=0
#
# ====================================
# SYM_EXTRACT TEST COMPLETED.
# ====================================
```

Hình 3.1. Kết quả transcript mô phỏng khối sym_extract



Hình 3.2. Waveform mô phỏng khối sym_extract

Kết quả mô phỏng cho thấy khối sym_extract hoạt động đúng theo các testcase đã xây dựng. Với đầu vào 16'hA5C3, khối xuất lần lượt các symbol 10, 10, 01, 01, 11, 00, 00, 11; với đầu vào 16'h8421, khối xuất 10, 00, 01, 00, 00, 10, 00, 01. Tín hiệu sym_cnt tăng từ 0 đến 7 trong quá trình tách dữ liệu, valid lên mức 1 khi dữ liệu hợp lệ và trở về 0 sau khi hoàn tất. Các dòng transcript đều báo PASS, xác nhận khối tách symbol đúng theo thiết kế.

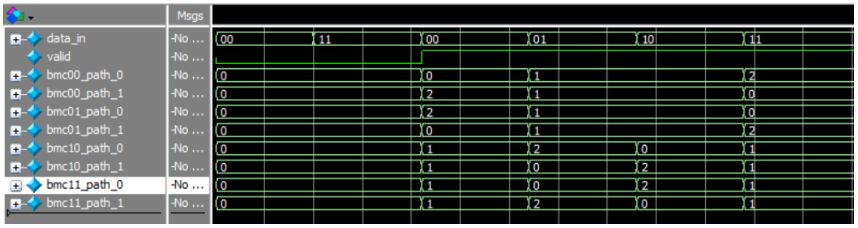
3.2. Khối bmu

Bảng 3.2. Testcase kiểm chứng khối bmu

Tên testcase	Mục đích	Điều kiện kiểm thử	Kết quả mong đợi
TC0: Valid gating	Kiểm tra tín hiệu valid	data_in = 2'b11, valid = 0	Tất cả branch metric = 0
TC1: Match 00	Kiểm tra BMU với data_in = 00	data_in = 2'b00, valid = 1	bmc = 0, 2, 2, 0, 1, 1, 1, 1
TC2: Match 01	Kiểm tra BMU với data_in = 01	data_in = 2'b01, valid = 1	bmc = 1, 1, 1, 1, 2, 0, 0, 2
TC3: Match 10	Kiểm tra BMU với data_in = 10	data_in = 2'b10, valid = 1	bmc = 1, 1, 1, 1, 0, 2, 2, 0
TC4: Match 11	Kiểm tra BMU với data_in = 11	data_in = 2'b11, valid = 1	bmc = 2, 0, 0, 2, 1, 1, 1, 1

```
# STARTING BMU TEST...
# [PASS]          TC0: Valid Gating Check | Data_in: 11
# [PASS]                    TC1: Match 00 | Data_in: 00
# [PASS]                    TC2: Match 01 | Data_in: 01
# [PASS]                    TC3: Match 10 | Data_in: 10
# [PASS]                    TC4: Match 11 | Data_in: 11
# BMU TEST COMPLETED.
```

Hình 3.3. Kết quả transcript mô phỏng khối bmu



Hình 3.4. Waveform mô phỏng khối bmu

Kết quả mô phỏng cho thấy khối bmu hoạt động đúng theo các testcase đã xây dựng. Khi valid = 0, toàn bộ các giá trị branch metric ở đầu ra đều bằng 0. Khi valid = 1, với các giá trị data_in lần lượt là 00, 01, 10 và 11, các tín hiệu bmc thay đổi đúng theo khoảng cách Hamming tương ứng.

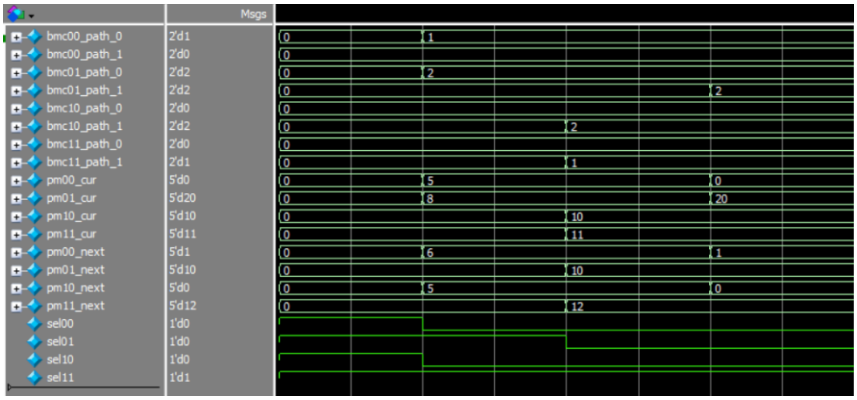
3.3. Khối acsu

Bảng 3.3. Testcase kiểm chứng khối acsu

Tên testcase	Mục đích	Điều kiện kiểm thử	Kết quả mong đợi
TC1: Normal Operation	Kiểm tra cộng và chọn nhánh có metric nhỏ hơn	pm00_cur = 5, bmc00_path_0 = 1; pm01_cur = 8, bmc01_path_0 = 2	Chọn nhánh 0; pm00_next = 6, sel00 = 0
TC2: Equal Metrics	Kiểm tra trường hợp hai metric bằng nhau	pm10_cur = 10, bmc10_path_1 = 2; pm11_cur = 11, bmc11_path_1 = 1	Hai candidate đều bằng 12; pm11_next = 12, sel11 ưu tiên nhánh 1
TC3: Init State Simulation	Kiểm tra chọn đường hợp lệ khi một path metric lớn	pm00_cur = 0, bmc00_path_1 = 0; pm01_cur = 20, bmc01_path_1 = 2	Chọn đường có metric nhỏ; pm10_next = 0, sel10 = 0

```
# === STARTING ACSU STANDALONE TEST ===
# Test 1: Normal Operation
# PASS Test 1
# Test 2: Equal Metrics
# State 11 Result: sel11=1, pm11_next=12
# Test 3: Init State Simulation (Check no 5-bit wrap issue)
# PASS Test 3
```

Hình 3.5. Transcript mô phỏng khối acsu



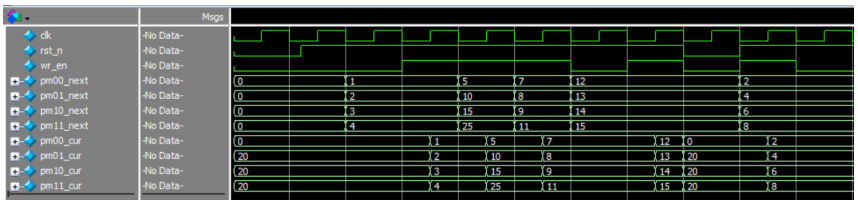
Hình 3.6. Waveform mô phỏng khối acsu

Kết quả mô phỏng cho thấy khối acsu hoạt động đúng với các testcase đã xây dựng.

3.4. Khối pmsm

Bảng 3.4. Testcase kiểm chứng khối pmsm

Tên testcase	Mục đích	Điều kiện kiểm thử	Kết quả mong đợi
TC1: Reset	Kiểm tra reset	rst_n = 0, wr_en = 0	pm_cur = 0, 20, 20, 20
TC2: Hold khi wr_en = 0	Kiểm tra giữ giá trị	wr_en = 0, input đổi thành 1, 2, 3, 4	pm_cur giữ giá trị sau reset
TC3: Ghi bộ metric 1	Kiểm tra ghi dữ liệu	wr_en = 1, pm_next = 1, 2, 3, 4	pm_cur = 1, 2, 3, 4
TC4: Ghi bộ metric 2	Kiểm tra ghi liên tiếp	wr_en = 1, pm_next = 5, 10, 15,25	pm_cur = 5, 10, 15, 25
TC5: Ghi bộ metric 3	Kiểm tra cập nhật liên tiếp	wr_en = 1, pm_next = 7, 8, 9, 11	pm_cur = 7, 8, 9, 11
TC6: Hold sau khi ghi	Kiểm tra không ghi khi wr_en tắt	wr_en = 0, input đổi thành 12, 13, 14, 15	pm_cur giữ 7, 8, 9, 11
TC7: Ghi lại sau hold	Kiểm tra ghi lại	wr_en = 1, pm_next = 12, 13, 14, 15	pm_cur = 12, 13, 14, 15
TC8: Reset sau khi ghi	Kiểm tra reset lại	rst_n = 0 sau khi đã ghi dữ liệu	pm_cur = 0, 20, 20, 20
TC9: Ghi sau reset	Kiểm tra hoạt động sau reset	rst_n = 1, wr_en = 1, pm_next = 2, 4, 6, 8	pm_cur = 2, 4, 6, 8
TC10: Tắt ghi	Kiểm tra trạng thái kết thúc	wr_en = 0	pm_cur giữ giá trị hiện tại



Hình 3.7. Waveform mô phỏng khối pmsm

Kết quả mô phỏng cho thấy khối pmsm hoạt động đúng theo các testcase đã xây dựng. Khi rst_n = 0, các path metric được khởi tạo về giá trị ban đầu 0, 20, 20, 20. Khi wr_en = 1, các giá trị pm_next được ghi vào các đầu ra pm_cur tại cạnh lên của clock. Khi wr_en = 0, khối giữ nguyên giá trị path metric hiện tại dù đầu vào thay đổi. Điều này xác nhận khối pmsm thực hiện đúng chức năng lưu trữ và cập nhật path metric.

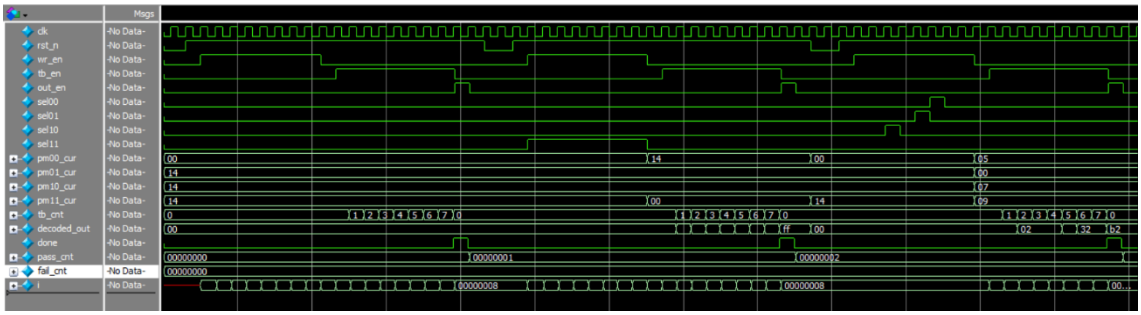
3.5. Khối smu_tbu

Bảng 3.5. Testcase kiểm chứng khối smu_tbu

Tên testcase	Mục đích	Điều kiện kiểm thử	Kết quả mong đợi
TC1: All-zero path	Kiểm tra traceback đường toàn 0	Ghi 8 chu kỳ survivor = 0; min_state = 00; tb_en chạy 8 chu kỳ, sau đó out_en = 1	decoded_out = 8'b00000000
TC2: All-one path	Kiểm tra traceback đường toàn 1	Ghi 8 chu kỳ với sel11 = 1; min_state = 11; tb_en chạy 8 chu kỳ, sau đó out_en = 1	decoded_out = 8'b11111111
TC3: Mixed path	Kiểm tra traceback với đường hỗn hợp	Ghi chuỗi decision bit theo t0-t7; min_state = 01; tb_en chạy 8 chu kỳ, sau đó out_en = 1	decoded_out = 8'b10110010

```
# [PASS] TEST_1_ALL_ZERO | decoded_out = 00000000 | expected = 00000000
# [PASS] TEST_2_ALL_ONE | decoded_out = 11111111 | expected = 11111111
# [PASS] 3_MIXED_10110010 | decoded_out = 10110010 | expected = 10110010
# ========================================
# TOTAL PASS = 3
# TOTAL FAIL = 0
# ========================================
```

Hình 3.8. Transcript mô phỏng khối smu_tbu



Hình 3.9. Waveform mô phỏng khối smu_tbu

Kết quả mô phỏng cho thấy khối smu_tbu hoạt động đúng theo các testcase đã xây dựng. Với trường hợp đường đi toàn 0, đầu ra giải mã là 00000000; với đường đi toàn 1, đầu ra là 11111111. Ở testcase đường hỗn hợp, khối traceback cho kết quả decoded_out = 10110010, đúng với giá trị mong đợi.

3.6. Khối ctrl_unit

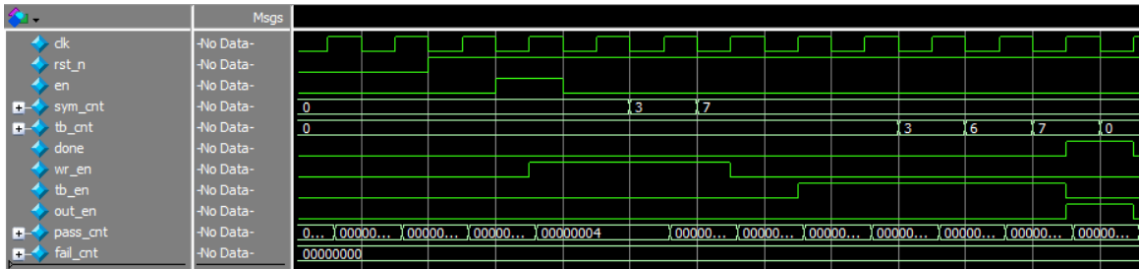
Bảng 3.6. Testcase kiểm chứng khối ctrl_unit

Tên testcase	Mục đích	Điều kiện kiểm thử	Kết quả mong đợi
TC1: Reset	Kiểm tra reset	rst_n = 0, en = 0	wr_en = 0, tb_en = 0, out_en = 0

TC2: Reset hold	Kiểm tra giữ reset	rst_n = 0 thêm 1 chu kỳ clock	wr_en = 0, tb_en = 0, out_en = 0
TC3: Idle	Kiểm tra trạng thái chờ	rst_n = 1, en = 0	wr_en = 0, tb_en = 0, out_en = 0
TC4: Process	Kiểm tra bắt đầu xử lý	en = 1, sym_cnt = 0, tb_cnt = 0	wr_en = 1, tb_en = 0, out_en = 0
TC5: Process hold	Kiểm tra giữ pha xử lý	en = 0, sym_cnt = 3 (<7)	wr_en = 1, tb_en = 0, out_en = 0
TC6: Wait	Kiểm tra kết thúc xử lý symbol	sym_cnt = 7	wr_en = 0, tb_en = 0, out_en = 0
TC7: Traceback	Kiểm tra bắt đầu traceback	Sau trạng thái WAIT 1 chu kỳ	wr_en = 0, tb_en = 1, out_en = 0
TC8: TB cnt 0	Kiểm tra giữ traceback	tb_cnt = 0	wr_en = 0, tb_en = 1, out_en = 0
TC9: TB cnt 3	Kiểm tra giữ traceback	tb_cnt = 3	wr_en = 0, tb_en = 1, out_en = 0
TC10: TB cnt 6	Kiểm tra giữ traceback	tb_cnt = 6	wr_en = 0, tb_en = 1, out_en = 0
TC11: Output	Kiểm tra xuất kết quả	tb_cnt = 7, done giả lập lên 1	wr_en = 0, tb_en = 0, out_en = 1
TC12: Back to idle	Kiểm tra quay về chờ	Sau pha OUT, tb_cnt = 0	wr_en = 0, tb_en = 0, out_en = 0

```
# [PASS] RESET | wr_en=0 tb_en=0 out_en=0 done=0
# [PASS] RESET_HOLD | wr_en=0 tb_en=0 out_en=0 done=0
# [PASS] IDLE | wr_en=0 tb_en=0 out_en=0 done=0
# [PASS] PROCESS | wr_en=1 tb_en=0 out_en=0 done=0
# [PASS] PROCESS_HOLD | wr_en=1 tb_en=0 out_en=0 done=0
# [PASS] WAIT | wr_en=0 tb_en=0 out_en=0 done=0
# [PASS] TB | wr_en=0 tb_en=1 out_en=0 done=0
# [PASS] TB_CNT_0 | wr_en=0 tb_en=1 out_en=0 done=0
# [PASS] TB_CNT_3 | wr_en=0 tb_en=1 out_en=0 done=0
# [PASS] TB_CNT_6 | wr_en=0 tb_en=1 out_en=0 done=0
# [PASS] OUT_BY_TB_CNT_7 | wr_en=0 tb_en=0 out_en=1 done=1
# [PASS] BACK_TO_IDLE | wr_en=0 tb_en=0 out_en=0 done=0
# ====================================
# TOTAL PASS = 12
# TOTAL FAIL = 0
# ====================================
```

Hình 3.10. Transcript mô phỏng khối ctrl_unit



Hình 3.11. Waveform mô phỏng khối ctrl_unit

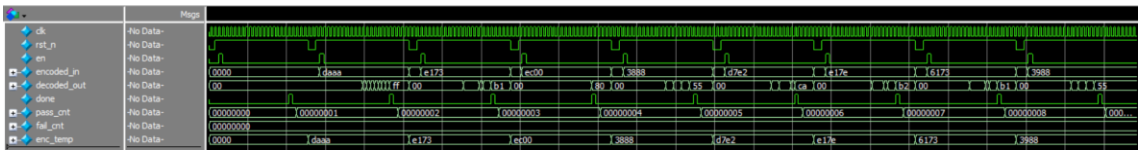
3.7. Mô phỏng toàn hệ thống viterbi_top

Bảng 3.7. Testcase cơ bản kiểm chứng khối viterbi_top

Tên testcase	Mục đích	Điều kiện kiểm thử	Kết quả mong đợi
TC1: All zeros	Kiểm tra giải mã chuỗi toàn 0	msg = 8'h00, encoded_in = 16'h0000, en = 1 trong 1 chu kỳ	done = 1, decoded_out = 8'h00 (00000000)
TC2: All ones	Kiểm tra giải mã chuỗi toàn 1	msg = 8'hFF, encoded_in = 16'hDAAA, en = 1 trong 1 chu kỳ	done = 1, decoded_out = 8'hFF (11111111)
TC3: 10110001	Kiểm tra mẫu dữ liệu hỗn hợp	msg = 8'hB1, encoded_in = 16'hE173, en = 1 trong 1 chu kỳ	done = 1, decoded_out = 8'hB1 (10110001)
TC4: 10000000	Kiểm tra mẫu có bit 1 ở MSB	msg = 8'h80, encoded_in = 16'hEC00, en = 1 trong 1 chu kỳ	done = 1, decoded_out = 8'h80 (10000000)
TC5: 01010101	Kiểm tra mẫu dữ liệu xen kẽ	msg = 8'h55, encoded_in = 16'h3888, en = 1 trong 1 chu kỳ	done = 1, decoded_out = 8'h55 (01010101)
TC6: 11001010	Kiểm tra mẫu dữ liệu hỗn hợp khác	msg = 8'hCA, encoded_in = 16'hD7E2, en = 1 trong 1 chu kỳ	done = 1, decoded_out = 8'hCA (11001010)
TC7: 10110010	Kiểm tra mẫu tham chiếu từ testbench khác	msg = 8'hB2, encoded_in = 16'hE17E, en = 1 trong 1 chu kỳ	done = 1, decoded_out = 8'hB2 (10110010)
TC8: 1-bit error	Kiểm tra sửa lỗi 1 bit ở symbol đầu	msg = 8'hB1, encoded_in = 16'h6173 sau khi đảo bit [15], en = 1 trong 1 chu kỳ	done = 1, decoded_out = 8'hB1 (10110001)
TC9: 1-bit error mid	Kiểm tra sửa lỗi 1 bit ở vị trí giữa	msg = 8'h55, encoded_in = 16'h3988 sau khi đảo bit [8], en = 1 trong 1 chu kỳ	done = 1, decoded_out = 8'h55 (01010101)

```
# ------------------------------------------------
#
# TC1: msg=00000000, encoded=0000000000000000 (0000)
# Latency: en -> done = 17 cycles, en -> decoded_out = 19 cycles
# [PASS] TC1 All zeros        : Encoded=0000, Decoded=00000000, Expected=00000000
#
# TC2: msg=11111111, encoded=1101101010101010 (daaa)
# Latency: en -> done = 17 cycles, en -> decoded_out = 19 cycles
# [PASS] TC2 All ones         : Encoded=daaa, Decoded=11111111, Expected=11111111
#
# TC3: msg=10110001, encoded=1110000101110011 (e173)
# Latency: en -> done = 17 cycles, en -> decoded_out = 19 cycles
# [PASS] TC3 10110001         : Encoded=e173, Decoded=10110001, Expected=10110001
#
# TC4: msg=10000000, encoded=1110110000000000 (ec00)
# Latency: en -> done = 17 cycles, en -> decoded_out = 19 cycles
# [PASS] TC4 10000000         : Encoded=ec00, Decoded=10000000, Expected=10000000
#
# TC5: msg=01010101, encoded=0011100010001000 (3888)
# Latency: en -> done = 17 cycles, en -> decoded_out = 19 cycles
# [PASS] TC5 01010101         : Encoded=3888, Decoded=01010101, Expected=01010101
#
# TC6: msg=11001010, encoded=1101011111100010 (d7e2)
# Latency: en -> done = 17 cycles, en -> decoded_out = 19 cycles
# [PASS] TC6 11001010         : Encoded=d7e2, Decoded=11001010, Expected=11001010
#
# TC7: msg=10110010, encoded=1110000101111110 (e17e)
# Latency: en -> done = 17 cycles, en -> decoded_out = 19 cycles
# [PASS] TC7 10110010         : Encoded=e17e, Decoded=10110010, Expected=10110010
#
# TC8: msg=10110001, encoded with 1-bit error=0110000101110011 (6173)
# Latency: en -> done = 17 cycles, en -> decoded_out = 19 cycles
# [PASS] TC8 1-bit error      : Encoded=6173, Decoded=10110001, Expected=10110001
#
# TC9: msg=01010101, encoded with 1-bit error=0011100110001000 (3988)
# Latency: en -> done = 17 cycles, en -> decoded_out = 19 cycles
# [PASS] TC9 1-bit error mid  : Encoded=3988, Decoded=01010101, Expected=01010101
#
# ================================================
#   RESULTS: 9 PASS, 0 FAIL
# ================================================
```

Hình 3.12. Transcript mô phỏng các test cơ bản của khối viterbi_top



Hình 3.13. Waveform mô phỏng các test cơ bản khối viterbi_top

Bên cạnh các testcase cơ bản, module viterbi_top còn được kiểm chứng bằng bộ dữ liệu đầu vào/đầu ra chuẩn được cung cấp.

```
# ==============================
# TEST 1021
# encoded_in   = 1101101010100011
# expected_out = 11111100
# Latency: en -> done = 17 cycles, en -> decoded_out = 19 cycles
# decoded_out  = 11111100
# => PASS
#
# ==============================
# TEST 1022
# encoded_in   = 1101101010100101
# expected_out = 11111100
# Latency: en -> done = 17 cycles, en -> decoded_out = 19 cycles
# decoded_out  = 11111100
# => PASS
#
# ==============================
# TEST 1023
# encoded_in   = 1101101010100110
# expected_out = 11111100
# Latency: en -> done = 17 cycles, en -> decoded_out = 19 cycles
# decoded_out  = 11111100
# => PASS
#
# ==============================
# TEST 1024
# encoded_in   = 1101101010100110
# expected_out = 11111100
# Latency: en -> done = 17 cycles, en -> decoded_out = 19 cycles
# decoded_out  = 11111100
# => PASS
#
# ==============================
# Tong test = 1025
# PASS      = 1025
# FAIL      = 0
```

Hình 3.14. Kết quả mô phỏng top module với bộ test đọc từ file input/output

Kết quả mô phỏng cho thấy module viterbi_top hoạt động đúng theo các testcase đã xây dựng. Ở nhóm test cơ bản, bộ giải mã khôi phục đúng dữ liệu gốc với nhiều dạng chuỗi đầu vào khác nhau, bao gồm chuỗi toàn 0, toàn 1, chuỗi xen kẽ và một số trường hợp có lỗi 1 bit trong chuỗi mã hóa. Ở nhóm test đọc dữ liệu từ file input.txt và output.txt, testbench tự động đưa từng chuỗi mã hóa 16 bit vào hệ thống, chờ tín hiệu done, sau đó so sánh decoded_out với kết quả mong đợi.

CHƯƠNG 4. THIẾT KẾ VẬT LÝ

4.1. Mục tiêu thiết kế

Giai đoạn thiết kế vật lý có nhiệm vụ hiện thực hóa bộ giải mã Viterbi từ mức netlist cổng logic sau tổng hợp sang dạng bố trí hình học vật lý trên layout, dựa trên thư viện công nghệ SkyWater 130nm (Sky130). Quá trình này nhằm đánh giá và tối ưu các chỉ số PPA gồm hiệu năng, công suất và diện tích, đồng thời kiểm tra độ tin cậy vật lý của thiết kế trước khi xuất file layout cuối cùng. Các mục tiêu chính của giai đoạn này gồm:

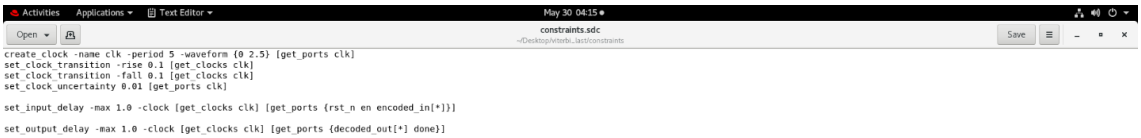
- **Hiệu năng (Performance):** Đảm bảo mạch có khả năng hoạt động tại tần số mục tiêu 200 MHz, tương ứng với chu kỳ clock 5 ns. Quá trình phân tích timing được thực hiện nhằm kiểm tra trạng thái timing closure, trong đó các ràng buộc setup time và hold time cần được đáp ứng trên các đường truyền quan trọng.
- **Công suất (Power):** Đánh giá công suất tiêu thụ của mạch sau tổng hợp và sau layout, bao gồm công suất động và công suất tĩnh. Kết quả power report được sử dụng để xác định thành phần tiêu thụ công suất chính và làm cơ sở cho các hướng tối ưu nếu cần.
- **Diện tích (Area):** Kiểm soát diện tích sử dụng của các standard cell và cấu hình mật độ sử dụng lõi ở mức hợp lý. Trong thiết kế này, core utilization được đặt khoảng 70% nhằm cân bằng giữa diện tích chip và khả năng đi dây, hạn chế nguy cơ nghẽn mạch trong giai đoạn routing.
- **Độ tin cậy vật lý (Physical Verification):** Kiểm tra layout sau khi hoàn thiện thông qua các bước kiểm tra như DRC, Connectivity và Antenna Check. Các kiểm tra này nhằm đảm bảo thiết kế tuân thủ quy tắc công nghệ, kết nối đúng với netlist và đủ điều kiện để xuất file GDSII/OASIS.

4.2. Tổng hợp logic (Synthesis) và chuẩn bị dữ liệu đầu vào cho layout

4.2.1. Vai trò của giai đoạn Synthesis

Giai đoạn tổng hợp logic được thực hiện nhằm chuyển mô tả RTL của bộ giải mã Viterbi thành gate-level netlist dựa trên thư viện công nghệ Sky130. Kết quả synthesis gồm netlist, file SDC và các báo cáo timing, area, power, được sử dụng làm dữ liệu đầu vào cho bước layout trên Cadence Innovus.

4.2.2. Thiết kế file ràng buộc constraints.sdc



Hình 4.1. File constraints.sdc

- Định nghĩa xung nhịp
  - create_clock -name clk -period 5 -waveform {0 2.5} [get_ports clk]: chu kỳ xung nhịp clock là 5ns tương ứng với chu kỳ T = 200MHz
- Độ dốc tín hiệu xung nhịp
  - set_clock_transition -rise 0.1 [get_clocks clk]
  - set_clock_transition -fall 0.1 [get_clocks clk]

Quy định thời gian để biên độ tín hiệu clock chuyển trạng thái Rise và Fall là 0.1ns (100ps)

- Sai số và độ bất định xung nhịp
  - set_clock_uncertainty 0.01 [get_ports clk]: Đặt biên độ sai số an toàn cho clock là 0.01ns (10ps), tương ứng với các hiện tượng như clock jitter, clock skew
- Ràng buộc thời gian input và output
  - set_input_delay -max 1.0 -clock [get_clocks clk] [get_ports {rst_n en encoded_in[*]}]: Khai báo các tín hiệu từ mạch ngoại vi bên ngoài truyền đến các chân input rst_n, en và encoded_in sẽ mất tối đa 1.0ns để ổn định kể từ cạnh lên của clock.
  - set_output_delay -max 1.0 -clock [get_clocks clk] [get_ports {decoded_out[*] done}]: Khai báo rằng khối mạch nhận tín hiệu ở bên ngoài chip yêu cầu dữ liệu xuất ra từ các chân decoded_out, done phải có sẵn và ổn định trước cạnh lên của xung nhịp tiếp theo ít nhất 1.0ns

4.2.3. Thiết lập file chạy tự động scripts genus.tcl

```
set_db init_lib_search_path ../lib/
set_db init_hdl_search_path ../rtl/

read_libs sky130_ss_1.62_125_nldm.lib

read_hdl {viterbi_top.v sym_extract.v bmu.v acsu.v pmsm.v smu_tbu.v ctrl_unit.v}

elaborate viterbi_top

set_db [get_db lib_cells *SDFF*] .dont_use true

read_sdc ../constraints/constraints.sdc

set_db syn_generic_effort medium
set_db syn_map_effort medium
set_db syn_opt_effort medium

syn_generic
syn_map
syn_opt

# Report
report_timing > reports/report_timing.rpt
report_area   > reports/report_area.rpt
report_power  > reports/report_power.rpt
report_qor    > reports/report_qor.rpt

# Output
write_hdl > outputs/viterbi_netlist.v
write_sdc > outputs/viterbi_sdc.sdc
```

Hình 4.2. File scripts genus.tcl

- Thiết lập đường dẫn tìm kiếm
  - set_db init_lib_search_path ../lib/: Khai báo thư mục chứa các file thư viện công nghệ (.lib) nằm ở thư mục ../lib/
  - set_db init_hdl_search_path ../rtl/: Khai báo thư mục chứa mã nguồn thiết kế phần cứng RTL (.v) nằm ở thư mục ../rtl/
- Nạp thư viện và Đọc mã nguồn RTL
  - read_libs sky130_ss_1.62_125_nldm.lib: Nạp file thư viện định thời (Timing Library) của công nghệ SkyWater 130nm ở góc chậm nhất (ss - slow-slow, điện áp thấp 1.62V, nhiệt độ cao $125°C$. Tool Genus dựa vào đây để biết độ trễ, công suất và diện tích của từng cổng logic (AND, OR, Flip-flop...)
  - read_hdl {viterbi_top.v sym_extract.v bmu.v acsu.v pmsm.v smu_tbu.v ctrl_unit.v}: Nạp toàn bộ các file mã nguồn Verilog của bộ giải mã Viterbi vào bộ nhớ của tool để chuẩn bị phân tích cú pháp
- Khai triển cấu trúc thiết kế (Elaboration)
  - elaborate viterbi_top: Lệnh này ra lệnh cho Genus phân tích kiến trúc phân cấp của mạch, lấy file viterbi_top làm khối Top module và tự động liên kết, kết nối các khối con bên trong (bmu, acsu, pmsm...) lại với nhau.
- Ràng buộc thiết kế (Constraint)

- set_db [get_db lib_cells *SDFF*] .dont_use true: Lệnh này ép Genus không được sử dụng các Scan Flip-Flop trong quá trình tổng hợp, nhằm tiết kiệm diện tích và tránh chèn nhầm các cell phục vụ test mạch (DFT) khi chưa cần thiết
- read_sdc ../constraints/constraints.sdc: Nạp file ràng buộc thiết kế constraints.sdc chứa các định nghĩa về tần số xung nhịp 200MHz, input/output delay,…

- Thiết lập mức độ nỗ lực tối ưu
  - set_db syn_generic_effort medium
  - set_db syn_map_effort medium
  - set_db syn_opt_effort medium

Thiết lập mức độ nỗ lực tính toán của thuật toán tối ưu ở cả 3 giai đoạn (Generic, Mapping, Optimization) ở mức **medium** (trung bình)

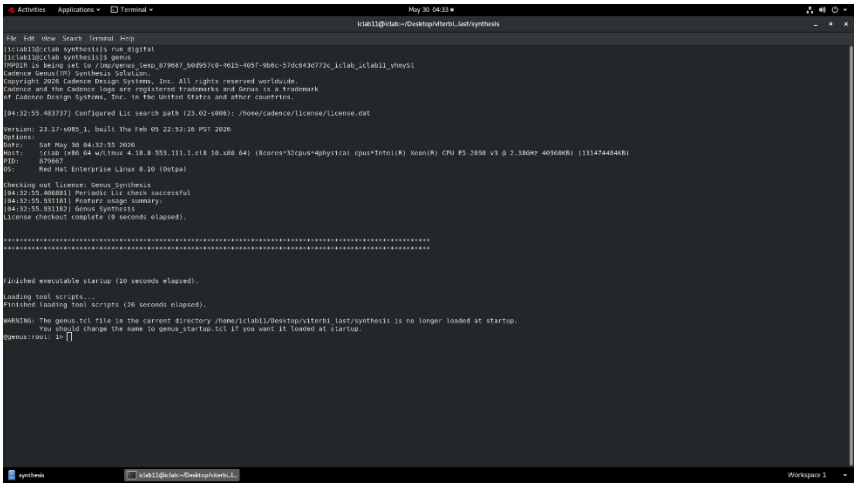
- Các giai đoạn tổng hợp cốt lõi
  - syn_generic: Biến đổi cấu trúc mạch trừu tượng từ bước elaborate thành cấu trúc mạng cổng logic chung độc lập với nhà máy đúc chip
  - syn_map: Ánh xạ các cổng logic chung thành các cổng logic vật lý cụ thể có trong thư viện mã sky130_ss_1.62_125_nldm.lib đã nạp ở bước đầu
  - syn_opt: Tool sẽ thực hiện tính toán định thời, chèn thêm Buffer, thay thế các cổng nhỏ bằng cổng lớn nhằm cố gắng sửa sạch lỗi Setup time, Hold time và thỏa mãn file SDC
- Xuất báo cáo
  - report_timing > reports/report_timing.rpt
  - report_area > reports/report_area.rpt
  - report_power > reports/report_power.rpt
  - report_qor > reports/report_qor.rpt

Chạy phân tích mạch và xuất các thông số kỹ thuật ra các file báo cáo tương ứng trong thư mục reports/:

- report_timing: Báo cáo thời gian trễ, cho biết Slack âm hay dương.
- report_area: Báo cáo diện tích tổng của các cell logic
- report_power: Báo cáo tổng công suất của mạch

- report_qor: Chất lượng thiết kế tổng quan (Quality of Results), thống kê số lượng cell, cấu trúc mạch

- Xuất dữ liệu đầu ra cho bước Layout
  - write_hdl > outputs/viterbi_netlist.v: Xuất ra file **Gate-level Netlist** (viterbi_netlist.v). Đây chính là file chứa kết nối các cổng logic của Sky130 dùng làm đầu vào để nạp vào tool Cadence Innovus làm Layout
  - write_sdc > outputs/viterbi_sdc.sdc: Xuất ra file ràng buộc định thời đã được công cụ chuẩn hóa lại để đồng bộ sang bước thiết kế vật lý.



Hình 4.3. Khởi chạy Cadence Genus và thực thi script genus.tcl

4.3. Kết quả sau Synthesis

4.3.1. Timing

```
============================================================
  Generated by:           Genus(TM) Synthesis Solution 23.17-s085_1
  Generated on:           May 30 2026  04:35:21 am
  Module:                 viterbi_top
  Operating conditions:   ss_1.62_125 (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
============================================================


Path 1: MET (336 ps) Setup Check with Pin u_pmsm/pm11_cur_reg[1]/CK->D
       Group: clk
    Startpoint: (R) u_sym_ext/data_in_reg[0]/CK
         Clock: (R) clk
      Endpoint: (F) u_pmsm/pm11_cur_reg[1]/D
         Clock: (R) clk

                  Capture       Launch
     Clock Edge:+    5000            0
    Src Latency:+       0            0
    Net Latency:+       0 (I)        0 (I)
        Arrival:=    5000            0

          Setup:-     166
    Uncertainty:-      10
  Required Time:=    4824
   Launch Clock:-       0
      Data Path:-    4488
          Slack:=     336
```

Hình 4.4. Báo cáo timing sau synthesis

Các thông số trong file report:

- Clock Edge: 5000ps – tương ứng chu kỳ T = 200MHz
- Trạng thái đường truyền: MET (Đạt yêu cầu)

- Setup time: 166ps
- Data Path: 4488ps
- Slack: 336ps (Dương)

4.3.2. Power

```
Instance: /viterbi_top
Power Unit: W
PDB Frames: /stim#0/frame#0
  ------------------------------------------------------------------------
    Category         Leakage     Internal    Switching        Total    Row%
  ------------------------------------------------------------------------
       memory     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
     register     5.95472e-07  6.08532e-04  7.97429e-05  6.88871e-04  48.60%
        latch     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
        logic     3.15452e-07  3.23534e-04  3.31786e-04  6.55635e-04  46.25%
         bbox     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
        clock     0.00000e+00  0.00000e+00  7.29583e-05  7.29583e-05   5.15%
          pad     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
           pm     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
  ------------------------------------------------------------------------
    Subtotal     9.10924e-07  9.32066e-04  4.84487e-04  1.41746e-03 100.00%
  Percentage           0.06%       65.76%       34.18%      100.00% 100.00%
  ------------------------------------------------------------------------
```

Hình 4.5. Báo cáo Power sau synthesis

Các thông số trong file report:

- Công suất nội tại (Internal Power): 9.32×10^{-4} W $\approx \mathbf{0.932}$ mW (Chiếm 65.76\%)
- Công suất đóng cắt (Switching Power): 4.84×10^{-4} W $\approx \mathbf{0.484}$ mW (Chiếm 34.18\%)
- Công suất rò rỉ (Leakage Power): 9.11×10^{-7} $W \approx \mathbf{0.911}$ $\mu\mathbf{W}$ (Chiếm 0.06\%)
- Tổng công suất của chip: 1.417 mW

4.3.3. Area

```
============================================================
  Generated by:           Genus(TM) Synthesis Solution 23.17-s085_1
  Generated on:           May 30 2026  04:35:21 am
  Module:                 viterbi_top
  Operating conditions:   ss_1.62_125 (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
============================================================

  Instance     Module    Cell-Count  Cell-Area  Net-Area   Total-Area   Wireload
-----------------------------------------------------------------------------------
viterbi_top NA                499   7251.912     0.000     7251.912 <none> (D)
  u_ctrl    ctrl_unit         18    239.955     0.000      239.955 <none> (D)
  u_pmsm    pmsm              40   1066.460     0.000     1066.460 <none> (D)
  u_smu     smu_tbu          201   3420.273     0.000     3420.273 <none> (D)
  u_sym_ext sym_extract       51    750.333     0.000      750.333 <none> (D)
  (D) = wireload is default in technology library
```

Hình 4.6. Báo cáo Area sau synthesis

- Tổng số lượng cổng logic là **499 cells**.
- Tổng diện tích toàn mạch đạt $\mathbf{7251.912um^2}$

4.4. Thiết kế Layout

4.4.1. Thiết lập file viterbi_mmmc.view

```
create_library_set -name max_timing\
    -timing ../lib/sky130_ss_1.62_125_nldm.lib

create_library_set -name min_timing\
    -timing ../lib/sky130_ff_1.98_0_nldm.lib

create_timing_condition -name default_mapping_tc_1\
    -library_sets max_timing
create_timing_condition -name default_mapping_tc_2\
    -library_sets min_timing

create_rc_corner -name rccorners\
    -pre_route_res 1\
    -pre_route_cap 1\
    -post_route_res 1\
    -post_route_cap 1\
    -post_route_cross_cap 1\
    -pre_route_clock_res 0\
    -pre_route_clock_cap 0\
    -qrc_tech ../qrc/qrcTechFile_RCgen

create_delay_corner -name max_delay\
    -timing_condition {default_mapping_tc_1}\
    -rc_corner rccorners
create_delay_corner -name min_delay\
    -timing_condition {default_mapping_tc_2}\
    -rc_corner rccorners

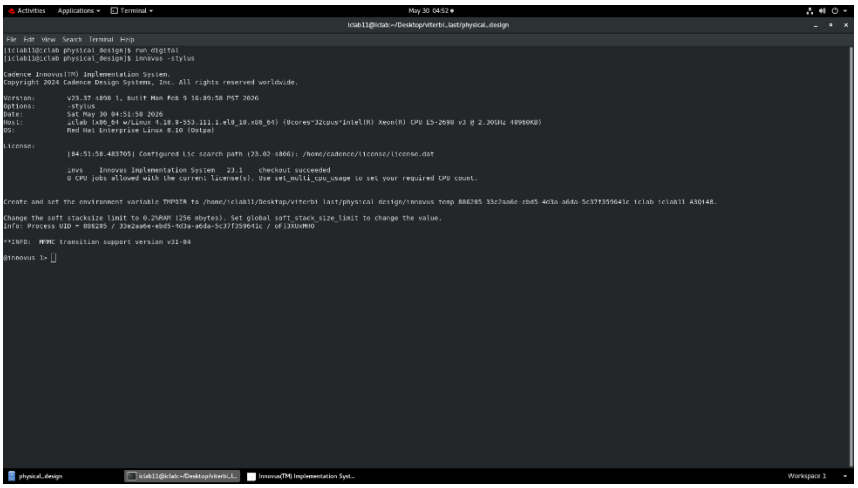
create_constraint_mode -name sdc_cons\
    -sdc_files ./viterbi_sdc.sdc

create_analysis_view -name wc -constraint_mode sdc_cons -delay_corner max_delay
create_analysis_view -name bc -constraint_mode sdc_cons -delay_corner min_delay

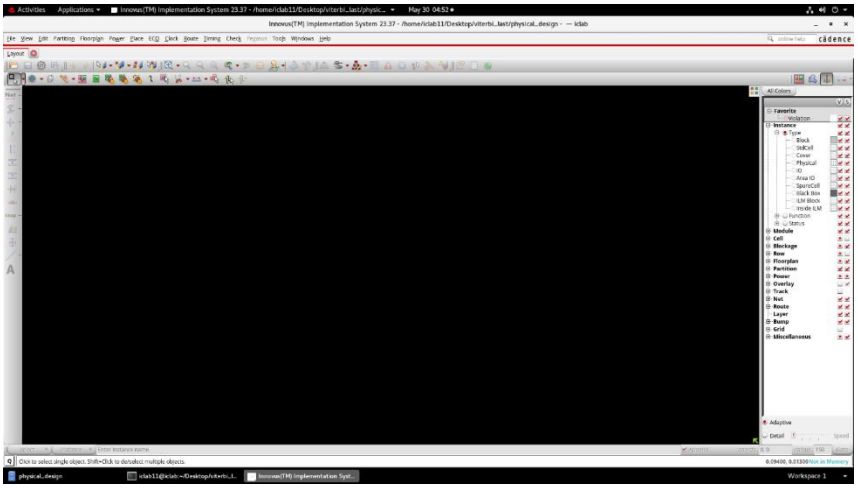

set_analysis_view -setup wc -hold bc
```

Hình 4.7. File viterbi_mmmc.view

4.4.2. Mở tool Cadence Innovus

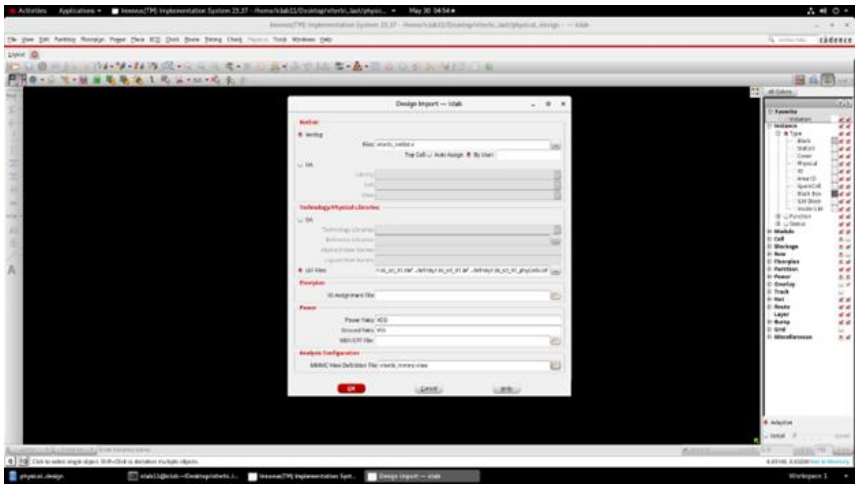


Hình 4.8. Khởi chạy Cadence Innovus



Hình 4.9. Giao diện GUI của Innovus

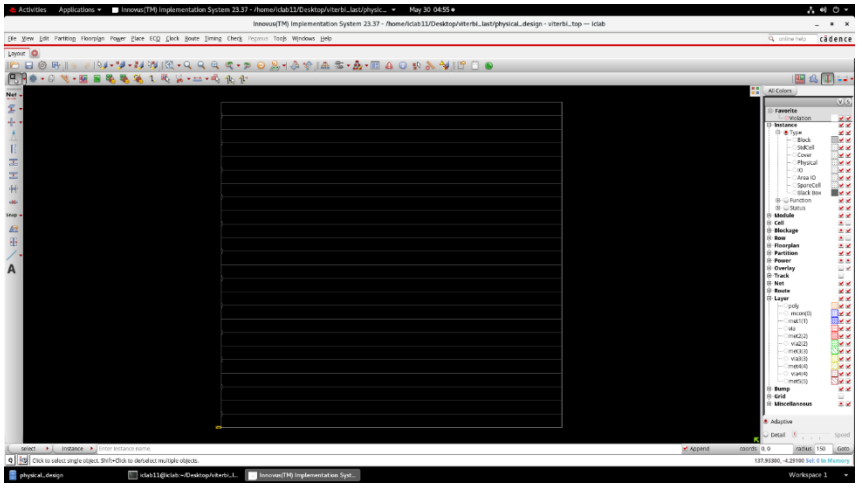
4.4.3. Import Design



Hình 4.10. Import Design

Import Design thực hiện nạp và đồng bộ hóa toàn bộ dữ liệu từ giai đoạn Synthesis sang môi trường Layout để chuẩn bị xây dựng chip vật lý.

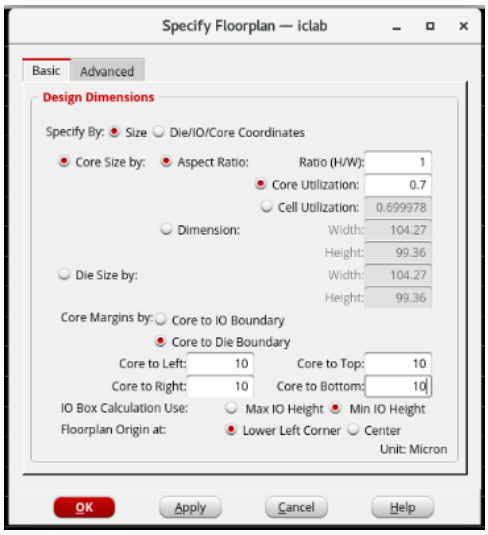
- **Netlist (viterbi_netlist.v):** Cung cấp cấu trúc kết nối logic (đề bài sơ đồ mạch gồm những cổng logic nào nối với nhau).
- **LEF Files (.tlef, .lef, _phyCells.lef):** Cung cấp thông tin vật lý hình học (kích thước của từng cổng, vị trí các chân cắm kim loại, luật đi dây của nhà máy SkyWater 130nm).
- **Power (VDD, VSS):** Định nghĩa tên cho mạng lưới cấp nguồn và đất toàn cục cho chip.
- **Analysis Configuration (viterbi_mmmc.view):** Nạp các kịch bản môi trường khắc nghiệt (Setup/Hold) và file ràng buộc thời gian SDC (tần số xung nhịp mục tiêu 200MHz).



Hình 4.11. Kết quả sau khi Import Design thành công

4.4.4. Floorplanning

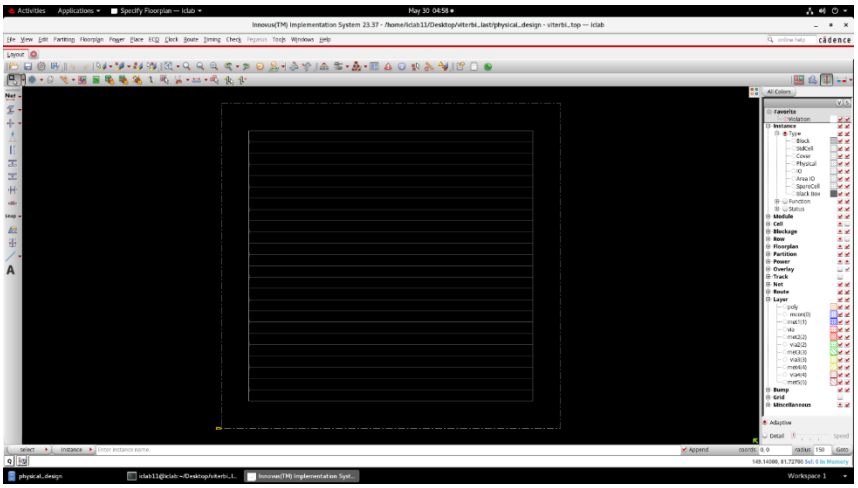
- Cấu hình kích thước chip



Hình 4.12. Setup cấu hình Floorplanning

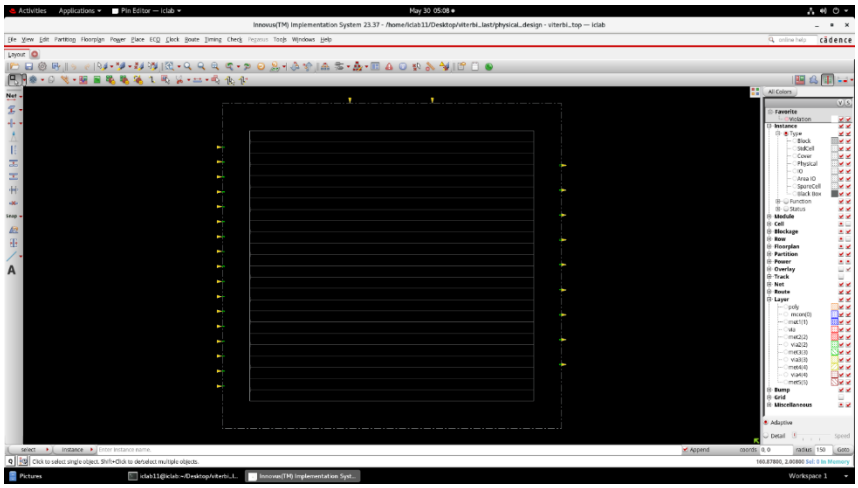
Floorplanning thực hiện khởi tạo hình học, định hình kích thước khuôn chip vật lý và phân chia ranh giới không gian cho khối viterbi_top trước khi đặt các linh kiện vào

- Aspect Ratio (H/W) = 1: cấu hình chip là hình vuông
- Core Utilization = 0.7: cấu hình diện tích của các Standard Cell của bộ viterbi chiếm 70% diện tích vùng Core



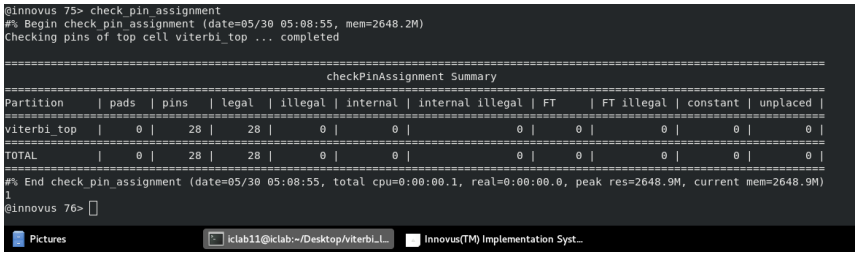
Hình 4.13. Kết quả sau khi Floorplanning

- Pin Editor



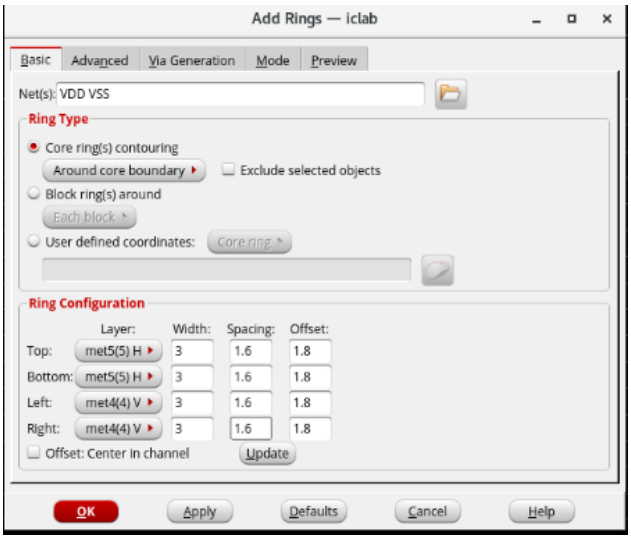
Hình 4.14. Cấu hình các pin của chip

Thực hiện định vị các chân I/O để chip giao tiếp với hệ thống bên ngoài



Hình 4.15. Thực hiện kiểm tra check_pin_assignment

- Add Ring

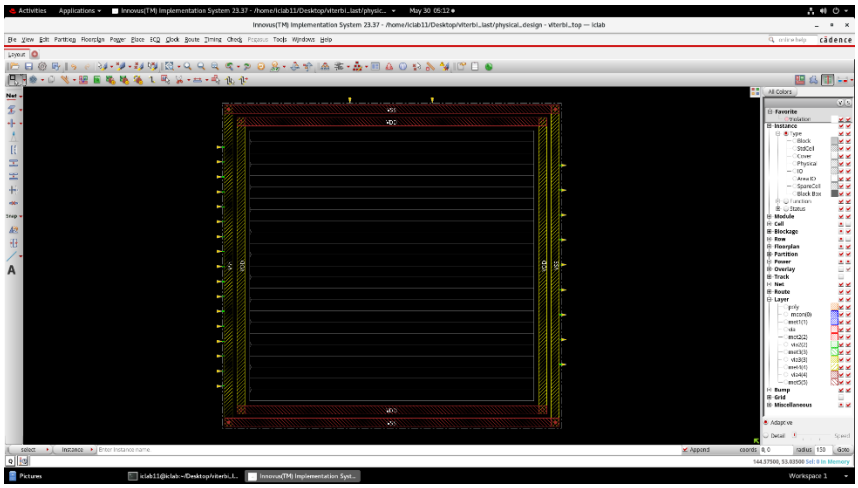


Hình 4.16. Add Ring

Add Ring thực hiện tạo mạng lưới phân phối nguồn chính bao quanh Core chip, cặp Ring đại diện cho hai mạng nguồn **VDD** và đất **VSS** được cấu hình phân tầng kim loại tối ưu theo quy tắc hướng đi dây của nhà sản xuất SkyWater: sử dụng

lớp kim loại cao **met5 (Horizontal)** cho hai cạnh ngang (Top/Bottom) và lớp **met4 (Vertical)** cho hai cạnh dọc (Left/Right).

Để đảm bảo khả năng gánh tải dòng điện lớn, giảm điện trở ký sinh và triệt tiêu hiện tượng sụt áp nội tại (*IR Drop*), bề rộng thanh nguồn (*Width*) được thiết là 3.0 um đi kèm khoảng cách cách ly an toàn giữa hai thanh (*Spacing*) là 1.6 um và khoảng cách lùi biên (*Offset*) đạt 1.8 um.



Hình 4.17. Kết quả sau khi Add Ring

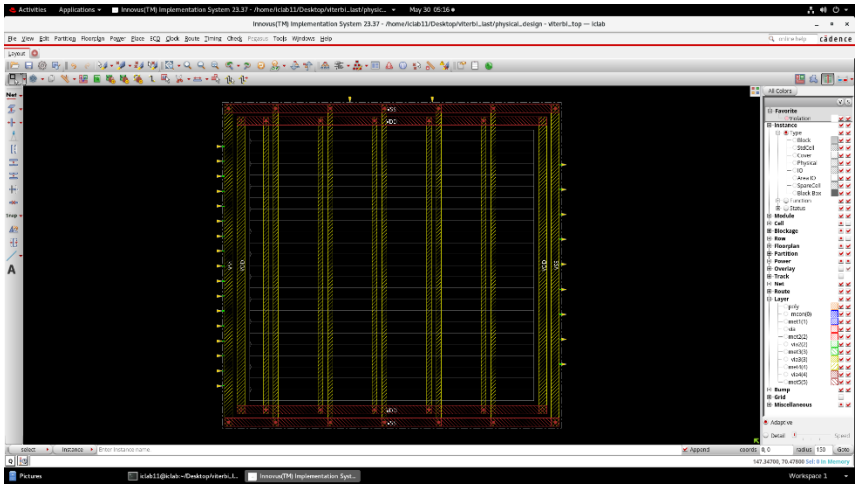
- Add Stripe



Hình 4.18. Add Stripes

Add Stripes thực hiện thiết lập các đường cao tốc năng lượng đâm xuyên trực tiếp vào Core của khối Viterbi. Mạng lưới cấp nguồn và đất (**VDD VSS**) được triển khai theo cấu trúc sọc dọc (**Vertical**) sử dụng lớp kim loại **met4** đồng bộ với hướng ưu tiên hình học của nhà máy SkyWater.

Để phân phối điện năng dồi dào và đồng đều tới mọi ngóc ngách của mạch logic, độ rộng của mỗi thanh nguồn (*Width*) được cấu hình ở mức **1.8 um** với khoảng cách cách ly an toàn (*Spacing*) đạt **1.8 um**. Cấu trúc sọc nguồn được thiết lập lặp lại tuần hoàn với khoảng cách giữa các cụm thanh (*Set-to-set distance*) cố định là **20 um**, xuất phát cách lề biên (*Start*) một khoảng **5 um**. Cấu hình này giúp triệt tiêu hoàn toàn rủi ro sụt áp cục bộ và chuẩn bị hạ tầng kết nối trực tiếp vào các hàng Standard Cells.

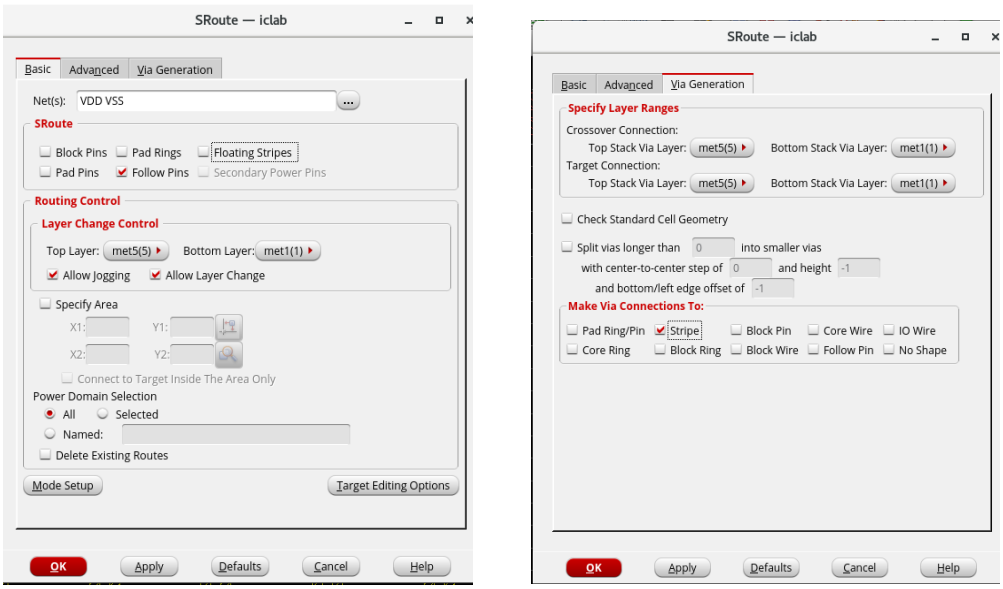


Hình 4.19. Kết quả sau khi Add Stripes

```
@innovus 80>
@innovus 80> connect_global_net VDD -type pg_pin -pin_base_name VDD -inst_base_name *
@innovus 81> connect_global_net VSS -type pg_pin -pin_base_name VSS -inst_base_name *
@innovus 82>
```

Hình 4.20. Kết nối chân VDD và VSS của mọi linh kiện

- Add Route



Hình 4.21. Add Route

Add Route thực hiện kéo các dây dẫn kim loại siêu mịn để cắm trực tiếp chân nguồn của từng cổng logic (Standard Cells) vào mạng lưới Stripes và Rings đã tạo.

- **Follow Pins = True**: Ra lệnh cho Innovus tự động tìm các đường ray nguồn kim loại mảnh (met1) chạy dọc theo mỗi hàng Standard Cell và kéo dài chúng ra để **cắm trực tiếp vào các thanh nguồn dọc (Stripes)**.
- **Make Via Connections To: Stripe = True**: Ra lệnh cho Innovus dập các lỗ xuyên tầng (**Vias**) tại mọi điểm giao cắt hình học giữa các hàng dây nguồn met1 và thanh dọc met4/met5 (Stripes)

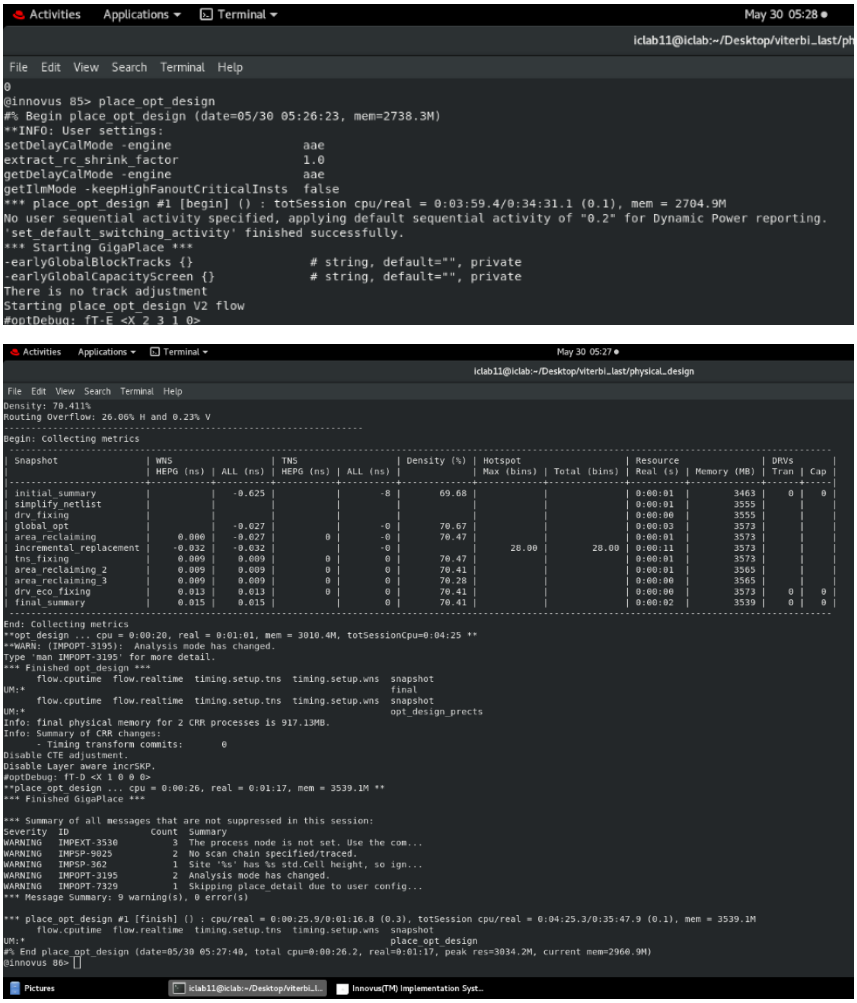


Hình 4.22. Kết quả sau khi Add Route

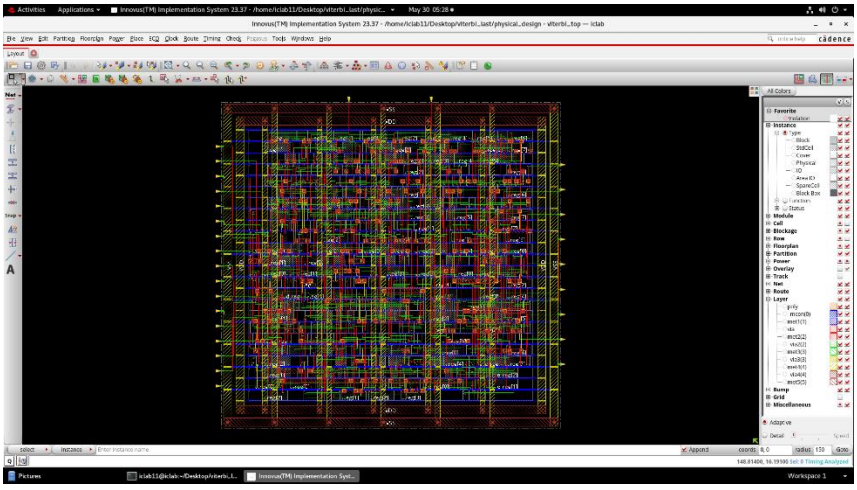
4.4.5. Placement

Câu lệnh place_opt_design thực hiện đồng thời hai nhiệm vụ: **Sắp đặt vị trí vật lý (Placement)** và **Tối ưu hóa định thời (Timing Optimization)** cho toàn bộ 499 cổng logic của bộ giải mã Viterbi.

Khi lệnh này khởi chạy, công cụ sẽ tự động phân bổ và cố định vị trí của các ô Standard Cells vào hệ thống các hàng (Rows) đã kẻ sẵn trong vùng lõi Core Box sao cho tổng chiều dài dây dẫn ngắn nhất nhằm giảm thiểu tụ điện ký sinh. Ngay sau đó, lõi tính toán định thời tích hợp sẽ rà soát toàn mạch; nếu phát hiện các đường truyền dữ liệu có nguy cơ bị chậm (vi phạm Setup Time), tool sẽ tự động thực hiện các biện pháp kỹ thuật như nhân bản cổng logic hoặc chèn thêm các ô đệm (Buffers) trung chuyển.



Hình 4.23. Thực hiện lệnh place_opt_design



Hình 4.24. Kết quả sau khi Placement

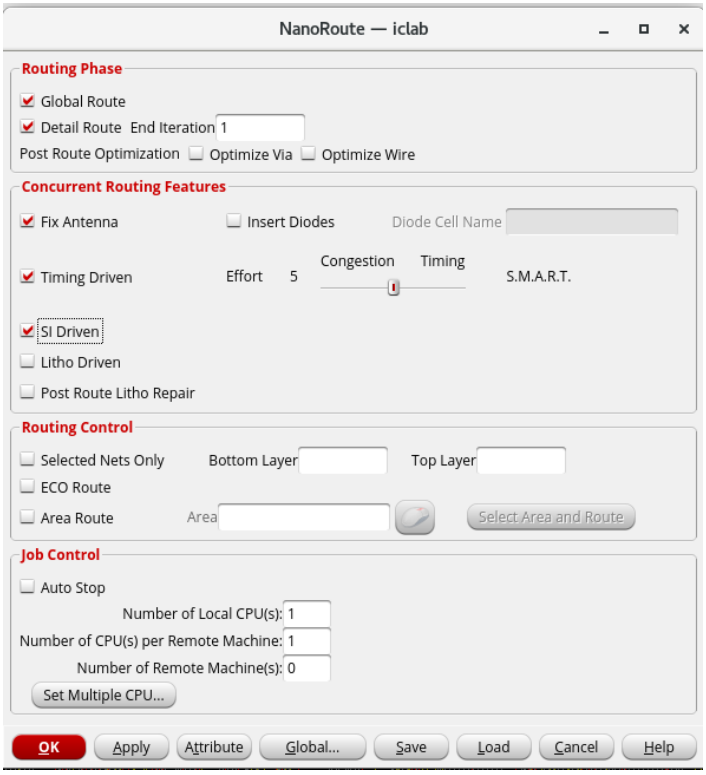
4.4.6. Clock Tree System – CTS

Thực hiện tổng hợp và tối ưu CTS cho khối Viterbi, giúp đưa tín hiệu xung nhịp 200MHz đến mọi thanh ghi, giảm thiểu tối đa Clock Skew



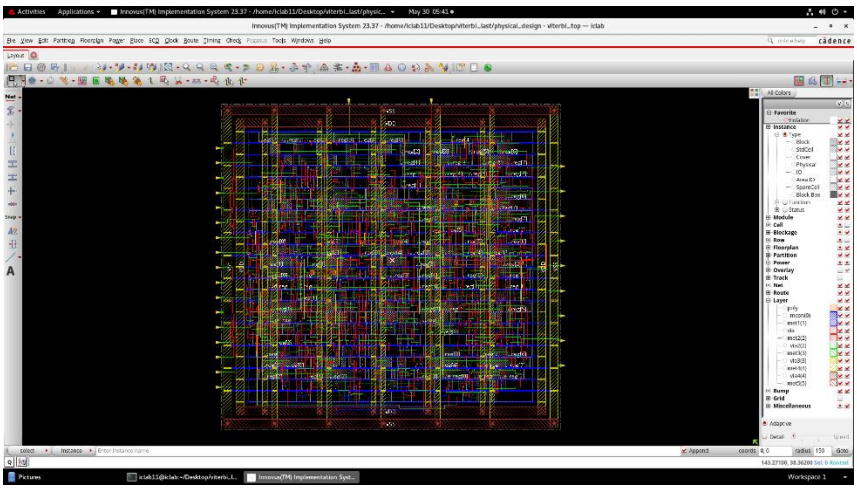
Hình 4.25. Tổng hợp và tối ưu CTS

4.4.7. Định tuyến chi tiết – Detail Route



Hình 4.26. Cấu hình NanoRoute

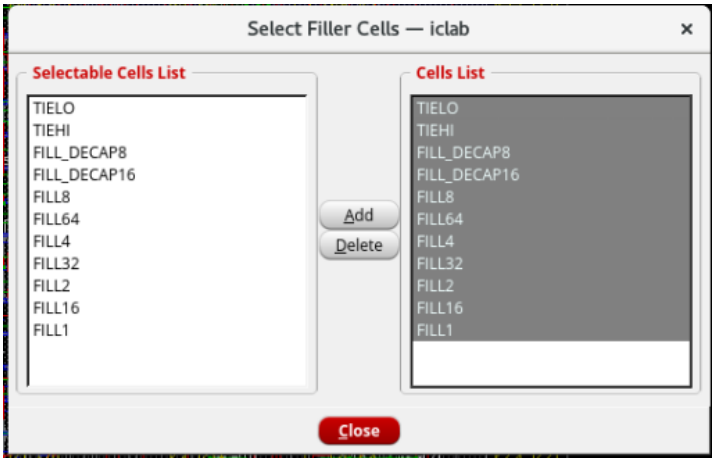
Thực hiện song song 2 chế độ: Timing Driven yêu cầu Innovus thực hiện các thuật toán ưu tiên đi dây thẳng nhất, ngắn nhất nhằm giảm điện trở và tụ ký sinh; SI Driven tính toán khoảng cách an toàn giữa các đường dây kim loại chạy song song, hạn chế tối đa hiện tượng nhiễu xuyên tâm (**Crosstalk**) và méo dạng sóng tín hiệu ở tần số cao.



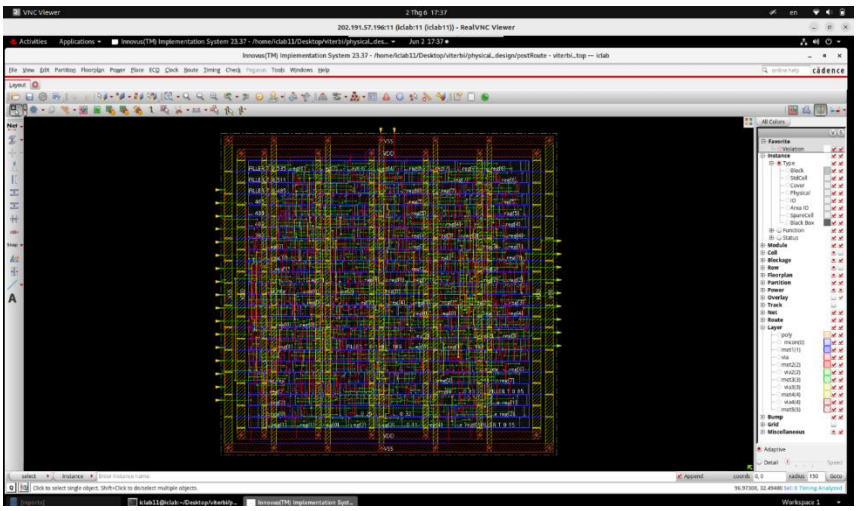
Hình 4.27. Kết quả sau khi NanoRoute

4.4.8. Add Filler

Thao tác **Add Filler** thực hiện bước lấp đầy các khoảng hở vật lý còn lại trên các hàng Standard Cell Rows bằng các ô đệm silicon thụ động (Filler Cells). Bước này mang hai ý nghĩa: một là tạo sự liên tục tuyệt đối cho dải bán dẫn nền dọc theo toàn mạch nhằm ổn định điện áp phân cực và triệt tiêu lỗi ngắn mạch Latch-up phá hủy chip; hai là đảm bảo mật độ phân bổ hình học đồng đều trên bề mặt theo đúng quy định của nhà máy SkyWater, ngăn hiện tượng lồi lõm khi mài phẳng wafer.



Hình 4.28. Add Filler



Hình 4.29. Kết quả sau khi Adder Filler

4.4.9. Check DRC, Connectivity, Antenna

```
@innovus 116> check_drc
#-check_same_via_cell true              # bool, default=false, user setting
 *** Starting Verify DRC (MEM: 3545.7) ***

  VERIFY DRC ..... Starting Verification
  VERIFY DRC ..... Initializing
  VERIFY DRC ..... Deleting Existing Violations
  VERIFY DRC ..... Creating Sub-Areas
  VERIFY DRC ..... Using new threading
  VERIFY DRC ..... Sub-Area: {0.000 0.000 124.660 119.600} 1 of 1
  VERIFY DRC ..... Sub-Area : 1 complete 0 Viols.

  Verification Complete : 0 Viols.

 *** End Verify DRC (CPU TIME: 0:00:00.1  ELAPSED TIME: 0:00:02.0  MEM: 0.0M) ***

1
@innovus 117>
Pictures  iclab11@iclab:~/Desktop/viterbi_l...  Innovus(TM) Implementation
```

Hình 4.30. Kết quả check_drc

```
@innovus 112> check_connectivity
VERIFY_CONNECTIVITY use new engine.

******** Start: VERIFY CONNECTIVITY ********
Start Time: Sat May 30 05:46:24 2026

Design Name: viterbi_top
Database Units: 1000
Design Boundary: (0.0000, 0.0000) (124.6600, 119.6000)
Error Limit = 1000; Warning Limit = 50
Check all nets

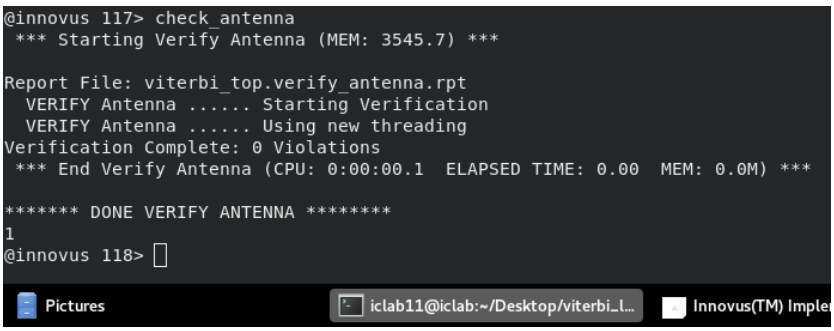
Begin Summary
  Found no problems or warnings.
End Summary

End Time: Sat May 30 05:46:24 2026
Time Elapsed: 0:00:00.0

******** End: VERIFY CONNECTIVITY ********
  Verification Complete : 0 Viols.  0 Wrngs.
  (CPU Time: 0:00:00.0  MEM: 0.000M)

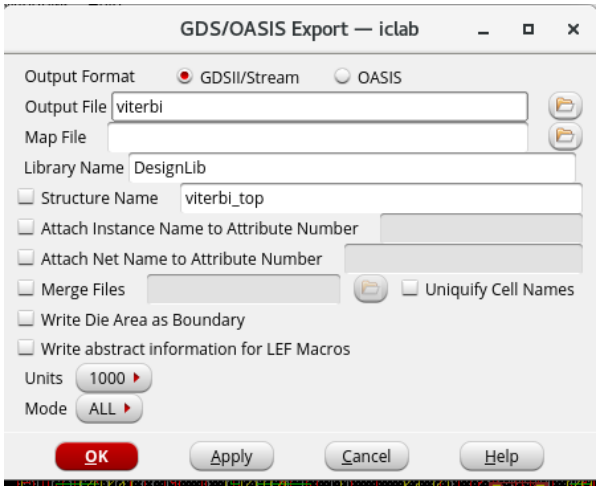
1
@innovus 113>
Pictures  iclab11@iclab:~/Desktop/viterbi_l...
```

Hình 4.31. Kết quả check_connectivity



Hình 4.32. Kết quả check_antenna

4.4.10. Xuất file GDSII/OASIS



Hình 4.33. Xuất file GDSII/OASIS

4.5. Kết quả sau Layout

4.5.1. Timing

```
#  Generated on:      Sat May 30 05:49:57 2026
#  Design:            viterbi_top
#  Command:           report_timing > reports/report_timing.rpt
#############################################################
Path 1: MET (0.061 ns) Setup Check with Pin u_pmsm/pm11_cur_reg[1]/CK->D
              View: wc
             Group: clk
        Startpoint: (R) u_sym_ext/data_in_reg[0]/CK
             Clock: (R) clk
          Endpoint: (F) u_pmsm/pm11_cur_reg[1]/D
             Clock: (R) clk

                     Capture       Launch
       Clock Edge:+    5.000        0.000
      Src Latency:+    0.000       -0.004
      Net Latency:+    0.004 (P)    0.004 (P)
          Arrival:=    5.004       -0.001

            Setup:-    0.220
      Uncertainty:-    0.010
    Required Time:=    4.774
     Launch Clock:=   -0.001
        Data Path:+    4.713
            Slack:=    0.061
```

Hình 4.34. Báo cáo Timing sau Layout

Các thông số:

- Trạng thái: MET (Đạt yêu cầu)
- Clock Edge: 5000ns tương ứng chu kỳ T = 200MHz
- Slack: 0.061ns

- Net Latency: 0.004ns (Độ trễ do xung nhịp chạy dọc theo cây Clock CTS thực tế đến chân kích của thanh ghi đích)
- Data Path: 4.713ns

Đánh giá tổng quan:

- Mạch đạt trạng thái Timing Closure thành công với mức Slack dương (+0.061ns) ở kịch abnr môi trường khắc nghiệt nhất

Khoảng dư 61ps là khá mong manh

4.5.2. Power

```
Total Power
--------------------------------------------------------------------------------------
Total Internal Power:        0.80261317            70.7447%
Total Switching Power:       0.33103458            29.1784%
Total Leakage Power:         0.00087309             0.0770%
Total Power:                 1.13452083
--------------------------------------------------------------------------------------


Group                           Internal   Switching     Leakage       Total  Percentage
                                Power      Power         Power         Power  (%)
--------------------------------------------------------------------------------------
Sequential                        0.6354     0.07368   0.0005928      0.7097       62.55
Macro                                  0          0           0           0           0
IO                                     0          0           0           0           0
Combinational                     0.1672     0.2574   0.0002803      0.4248       37.45
Clock (Combinational)                  0          0           0           0           0
Clock (Sequential)                     0          0           0           0           0
--------------------------------------------------------------------------------------
Total                             0.8026      0.331   0.0008731       1.135         100
--------------------------------------------------------------------------------------


Rail                  Voltage   Internal   Switching     Leakage       Total  Percentage
                                Power      Power         Power         Power  (%)
--------------------------------------------------------------------------------------
VDD                      1.62     0.8026      0.331   0.0008731       1.135         100


--------------------------------------------------------------------------------------
*       Power Distribution Summary:
*              Highest Average Power:           FE_OFC18_rst_n (BUFX2):          0.01339
*              Highest Leakage Power:       u_sym_ext/valid_reg (DFFRX1):        7.628e-06
*                Total Cap:      5.30173e-12 F
*                Total instances in design:   501
*                Total instances in design with no power:     0
*                Total instances in design with no activty:     0

*                Total Fillers and Decap:     0
--------------------------------------------------------------------------------------

** INFO:  (VOLTUS_POWR-3465): There are 0 decaps and 0 fillers in the design

Ended Static Power Report Generation: (cpu=0:00:00, real=0:00:00,
mem(process/total/peak)=3131.08MB/5778.13MB/3131.08MB)
```

Hình 4.35. Báo cáo Power sau Layout

Các thông số:

- Tổng công suất tiêu thụ toàn mạch sau khi đi dây hoàn chỉnh đạt **1.135 mW**
  - Công suất nội tại (Internal Power): 0.803 mW (Chiếm 70.74%)
  - Công suất đóng cắt đường dây (Switching Power): 0.331 mW (Chiếm 29.18%)
  - Công suất rò rỉ tĩnh (Leakage Power): 0.00087 mW $\approx \mathbf{0.87}\ \mu\mathbf{W}$ (Chiếm 0.077%)
- Nhóm linh kiện
  - **Sequential (Chiếm 62.55%):** Gồm 24 cụm thanh ghi trạng thái chiếm phần lớn công suất do các Flip-flop hoạt động liên tục theo xung nhịp 200MHz, phát sinh Internal Power rất lớn (0.6354 mW)

- **Combinational (Chiếm 37.45%):** Gồm 19 tầng logic tính toán khoảng cách toán học, tiêu tốn năng lượng chủ yếu ở phần **Switching Power** (0.2574 mW) do các đường dây kết nối đan chéo giữa các cổng logic liên tục đảo trạng thái lật bit 0 và 1

Đánh giá tổng quan:

- Tổng công suất tiêu thụ dừng ở mức **1.135 mW** là một kết quả khá ổn cho một lõi giải mã Viterbi xử lý thuật toán phức tạp chạy ở tần số cao.
- Việc dòng điện hoạt động ổn định trên đường ray nguồn VDD (1.62V) mà không gây ra bất kỳ cảnh báo nào về quá tải dòng cho thấy mạng lưới đai nguồn *Add Rings* và *Add Stripes* thiết kế có độ dãn tải và chịu dòng dư dả.

4.5.3. Area

Hinst Name	Module Name	Inst Count	Total Area
viterbi_top		501	7320.514
u_ctrl	ctrl_unit	18	243.763
u_pmsm	pmsm	40	1066.464
u_smu	smu_tbu	197	3416.494
u_sym_ext	sym_extract	51	750.334

Hình 4.36. Báo cáo Area sau Layout

Các thông số:

- **Số lượng cổng (Inst Count): 501.** Việc tăng thêm 2 cell so với synthesis là kết quả của lệnh place_opt_design và clock_opt_design. Công cụ đã chủ động chèn thêm các cổng đệm vật lý (như ô BUFX2 khuếch đại đường reset hệ thống) nhằm sửa dốc tín hiệu (Transition) và cân bằng mạng lưới cây xung nhịp CTS.
- **Tổng diện tích (Total Area): 7320.514 µm²** (Tăng thêm **68.602 µm²** so với mức 7251.912 µm² ở bước Synthesis).

Đánh giá tổng quan:

- Diện tích tăng 0.9% so với synthesis cho thấy mã nguồn RTL khá chặt chẽ, không bắt buộc công cụ phải chèn quá nhiều cell đệm sửa lỗi (Buffer/Inverter).
- Khối có diện tích lớn nhất là Survivor Memory and Traceback Unit (smu_tbu)

KẾT LUẬN

Trong bài tập lớn này, nhóm đã thực hiện thiết kế bộ giải mã Viterbi cho mã tích chập có độ dài ràng buộc (K = 3), tốc độ mã (R = 1/2), dữ liệu đầu vào 16 bit và dữ liệu đầu ra 8 bit. Thiết kế được mô tả bằng Verilog HDL và triển khai theo quy trình thiết kế mạch số VLSI, bao gồm xây dựng kiến trúc phần cứng, viết RTL, tạo testbench kiểm chứng, tổng hợp và thiết kế vật lý bằng các công cụ Cadence.

Về mặt kiến trúc, hệ thống được chia thành các khối chức năng chính gồm sym_extract, bmu, acsu, pmsm, smu_tbu và control_unit. Các khối này phối hợp để thực hiện các bước xử lý của thuật toán Viterbi như tách symbol 2 bit, tính branch metric, cập nhật path metric, lưu decision bit và traceback để khôi phục dữ liệu gốc. Việc phân chia thành các module riêng giúp thiết kế rõ ràng, thuận lợi cho mô phỏng từng khối và tích hợp ở mức top module.

Ở mức RTL, nhóm đã xây dựng testbench cho từng khối con và cho toàn bộ hệ thống. Các testcase tập trung kiểm tra những chức năng chính như reset, enable, xử lý dữ liệu, lưu trạng thái và truy vết ngược. Kết quả transcript và waveform cho thấy các khối hoạt động đúng theo đặc tả đã xây dựng. Đối với mô phỏng top module, testbench đọc dữ liệu từ file input/output và kiểm tra tổng cộng 1025 mẫu dữ liệu. Kết quả thu được là 1025 mẫu PASS và 0 mẫu FAIL, cho thấy đầu ra decoded_out khớp với dữ liệu mong đợi trong tập kiểm thử.

Sau khi hoàn thành mô phỏng RTL, thiết kế được tổng hợp bằng Cadence Genus để tạo netlist mức cổng và tiếp tục được kiểm chứng ở mức sau tổng hợp. Kết quả phân tích timing sau synthesis cho thấy thiết kế có slack dương 336 ps, tức là chưa vi phạm setup timing trong điều kiện ràng buộc đang xét.

Ở giai đoạn thiết kế vật lý, nhóm sử dụng Cadence Innovus để thực hiện floorplan, placement, clock tree synthesis và routing. Sau layout, thiết kế tiếp tục được mô phỏng để kiểm tra lại chức năng khi xét đến trễ sau triển khai. Kết quả cho thấy thiết kế vẫn duy trì đúng chức năng giải mã theo các testcase kiểm chứng.

Nhìn chung, bộ giải mã Viterbi đã được kiểm chứng qua các mức RTL, sau tổng hợp và sau layout. Thiết kế hoạt động đúng chức năng trên tập dữ liệu kiểm thử và có thể tiếp tục được tối ưu thêm về độ trễ, diện tích và công suất.